Métodos de Ordenação

1. O que é ordenar

Ordenação é o processo de reorganizar um conjunto de dados em uma ordem **ascendente** (crescente) ou **descendente** (decrescente), segundo algum **critério**. O critério pode ser o valor de um número, a ordem alfabética de um nome, o preço de um produto, a data de um pedido etc.

O objetivo da ordenação é **facilitar a recuperação posterior** dos itens. Na aula passada vimos um exemplo: a busca binária é muito mais rápida que a linear, mas só funciona em uma lista **ordenada**. Ordenar é o que torna essa busca possível.

Ordenar (assim como buscar) é um dos problemas mais básicos da computação. Alguns exemplos de aplicação:

- criação de **rankings** (os mais vendidos, a pontuação de um jogo);
- **listas ordenadas** (contatos do celular, extrato bancário por data);
- **otimização de buscas**, como a busca binária.

A maioria dos métodos de ordenação é baseada em **comparações** entre os itens. Por convenção, nesta aula vamos ordenar **listas de números**, mas os mesmos algoritmos servem para qualquer sequência cujos itens possam ser comparados (nomes, ou até dicionários, comparando um campo, como no exercício 5).

Existem dois grupos de métodos:

	Métodos simples	Métodos sofisticados
Indicados para	Conjuntos **pequenos** de dados	Conjuntos **grandes** de dados
Lógica	Simples, fácil de implementar	Mais elaborada, implementação mais trabalhosa
Tempo de execução	**Maior**	**Menor**
Métodos desta aula	Bubble sort, Selection sort, Insertion sort	Merge sort, Quick sort

Um detalhe importante: os algoritmos desta aula **ordenam a própria lista** (a lista recebida é modificada) e, por isso, **não devolvem nada**. Veja o que acontece se alguém tentar usar o resultado:

```
lista = [3, 1, 2]
lista = bubble_sort(lista)      # ERRO de lógica: bubble_sort devolve None
print(lista)                    # None  (a lista foi perdida!)
```

O jeito certo é chamar a função e depois usar a lista, que já foi ordenada:

```
lista = [3, 1, 2]
bubble_sort(lista)
print(lista)                    # [1, 2, 3]
```

2. Como medir o custo de ordenar

Como na busca, o custo de um algoritmo de ordenação é medido pela quantidade de **operações críticas** que ele efetua. Em ordenação, são duas:

- a **comparação** de dois itens (`lista[j] > lista[j + 1]`, por exemplo);
- a **troca de posição** entre dois itens.

Quanto menos operações críticas, mais rápido é o algoritmo. Há duas formas de medir:

1. **Contar** as operações (como fizemos com as comparações na Aula 13). O número não depende do computador.
2. **Cronometrar** o tempo de execução com `datetime` (também da Aula 13). O tempo depende do computador, mas mostra o efeito real.

2.1 A troca de dois itens

Quase todos os algoritmos desta aula precisam **trocar** dois itens de lugar. Para trocar os valores de `lista[j]` e `lista[j + 1]` é preciso uma variável auxiliar, `aux`:

```
aux = lista[j]              # guarda o valor de lista[j]
lista[j] = lista[j + 1]     # lista[j] recebe o valor do vizinho
lista[j + 1] = aux          # o vizinho recebe o valor que estava guardado
```

Por que a variável `aux`? Veja o que acontece **sem** ela, com `lista = [5, 3]` e `j = 0`:

Passo	Comando	`lista` depois	O que acontece
1	`lista[0] = lista[1]`	`[3, 3]`	O `5` foi **apagado** (sobrescrito pelo 3)
2	`lista[1] = lista[0]`	`[3, 3]`	Copia o 3 de volta: os dois ficaram iguais, e o `5` se perdeu

Com `aux`: `aux = 5`, depois `lista[0] = 3`, depois `lista[1] = aux`, que vale `5`: a lista vira `[3, 5]`.

O Python também permite trocar em uma única linha, com a atribuição múltipla: `lista[j], lista[j + 1] = lista[j + 1], lista[j]`. Nesta aula usaremos o `aux`, porque deixa cada passo da troca visível.

3. Bubble sort (ordenação por bolha)

3.1 A ideia

O **bubble sort** é um dos algoritmos de ordenação mais lentos, mas tem implementação simples. Funciona comparando os itens **dois a dois** (vizinhos) e consiste em "borbulhar" o maior item até o final da lista, como uma bolha que sobe.

Uma **varredura** funciona assim:

1. compara o 1º item com o 2º e troca os dois se o 1º for **maior**;
2. compara o 2º com o 3º e troca se o 2º for maior;
3. compara o 3º com o 4º e troca se o 3º for maior;
4. e assim por diante, até o fim da lista.

Ao final de uma varredura, o **maior item está no final** da lista, no seu lugar definitivo. Então repete-se o processo: a 2ª varredura coloca o 2º maior no penúltimo lugar, e assim por diante. Como o final da lista já está certo, cada varredura precisa olhar **um par a menos** que a anterior. Para uma lista de `N` itens, bastam `N - 1` varreduras (o último item que sobra já está no lugar).

3.2 Exemplo passo a passo

Vamos ordenar `[5, 3, 2, 1, 90, 6]`. Nos quadros, o par que está sendo comparado aparece em **negrito**, e a coluna "Lista depois" mostra a lista logo depois da comparação (e da troca, se houve). A barra `|`, quando aparece, separa a parte que já está no lugar definitivo.

Varredura 1 (`i = 0`, compara os pares até a posição 5):

Passo	j	Comparação	Resultado	Lista depois
1	0	`5 > 3`	Verdadeiro: **troca**	[**3, 5**, 2, 1, 90, 6]
2	1	`5 > 2`	Verdadeiro: **troca**	[3, **2, 5**, 1, 90, 6]
3	2	`5 > 1`	Verdadeiro: **troca**	[3, 2, **1, 5**, 90, 6]
4	3	`5 > 90`	Falso: não troca	[3, 2, 1, **5, 90**, 6]
5	4	`90 > 6`	Verdadeiro: **troca**	[3, 2, 1, 5, **6, 90**]

Fim da varredura 1: o `90` ficou no lugar definitivo. Lista: [3, 2, 1, 5, 6 | 90]

- Exemplo: `[5, 3, 2, 1, 90, 6]`
- 1ª Iteração:

  `[5,3,2,1,90,6]`
  `[3,5,2,1,90,6]`
  `[3,2,5,1,90,6]`
  `[3,2,1,5,90,6]`
  `[3,2,1,5,90,6]`
  `[3,2,1,5,6,90]` Ao final da 1ª iteração, o maior estará no final
- *Agora temos que repetir o processo novamente*

Varredura 1: em vermelho, o par comparado em cada passo; em azul, o `90`, que terminou no lugar definitivo.

Varredura 2 (`i = 1`, compara os pares até a posição 4):

Passo	j	Comparação	Resultado	Lista depois
1	0	`3 > 2`	Verdadeiro: **troca**	[**2, 3**, 1, 5, 6, 90]
2	1	`3 > 1`	Verdadeiro: **troca**	[2, **1, 3**, 5, 6, 90]
3	2	`3 > 5`	Falso: não troca	[2, 1, **3, 5**, 6, 90]
4	3	`5 > 6`	Falso: não troca	[2, 1, 3, **5, 6**, 90]

Fim da varredura 2: o `6` ficou no lugar definitivo. Lista: [2, 1, 3, 5 | 6, 90]

- Exemplo: `[5, 3, 2, 1, 90, 6]`
- 2ª Iteração:

  `[3,2,1,5,6,90]`
  `[2,3,1,5,6,90]`
  `[2,1,3,5,6,90]`
  `[2,1,3,5,6,90]` Ao final da 2ª iteração, o segundo maior estará no seu lugar
- *Agora temos que repetir o processo novamente*

Varredura 2: o `6` terminou no lugar definitivo (o `90` já estava).

Varredura 3 (`i = 2`, compara os pares até a posição 3):

Passo	j	Comparação	Resultado	Lista depois
1	0	`2 > 1`	Verdadeiro: **troca**	[**1, 2**, 3, 5, 6, 90]
2	1	`2 > 3`	Falso: não troca	[1, **2, 3**, 5, 6, 90]
3	2	`3 > 5`	Falso: não troca	[1, 2, **3, 5**, 6, 90]

Fim da varredura 3: o `5` ficou no lugar definitivo. Lista: [1, 2, 3 | 5, 6, 90]

- Exemplo: `[5, 3, 2, 1, 90, 6]`
- 3ª Iteração:

  `[2,1,3,5,6,90]`

  `[1,2,3,5,6,90]`

  `[1,2,3,5,6,90]`

  `[1,2,3,5,6,90]`

  `[1,2,3,5,6,90]` Ao final da 3ª iteração, o terceiro maior estará no seu lugar
- *Agora temos que repetir o processo novamente*

Varredura 3: o `5` terminou no lugar definitivo.

Varredura 4 (`i = 3`, compara os pares até a posição 2):

Passo	j	Comparação	Resultado	Lista depois
1	0	`1 > 2`	Falso: não troca	[**1, 2**, 3, 5, 6, 90]
2	1	`2 > 3`	Falso: não troca	[1, **2, 3**, 5, 6, 90]

Fim da varredura 4: o `3` ficou no lugar definitivo. Lista: [1, 2 | 3, 5, 6, 90]

- Exemplo: `[5, 3, 2, 1, 90, 6]`
- 4ª Iteração:

  `[1,2,3,5,6,90]`

  `[1,2,3,5,6,90]`

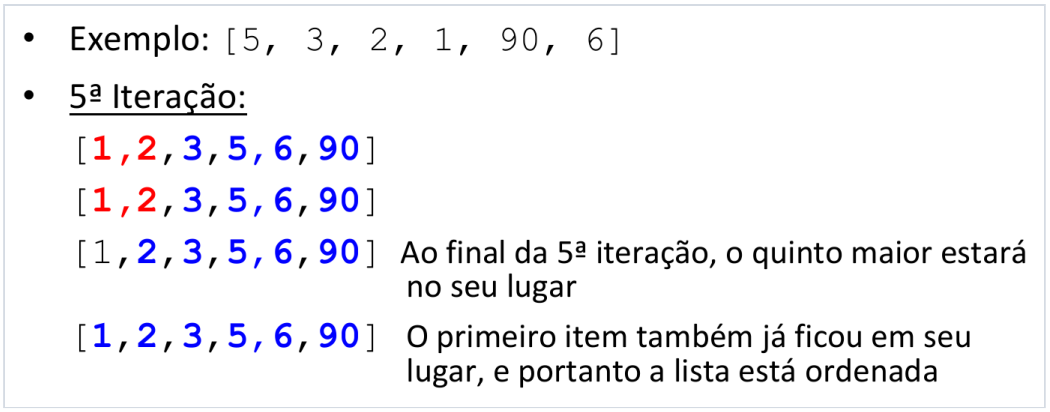
  `[1,2,3,5,6,90]` Ao final da 4ª iteração, o quarto maior estará no seu lugar
- *Agora temos que repetir o processo novamente*

Varredura 4: o `3` terminou no lugar definitivo.

Varredura 5 (`i = 4`, compara os pares até a posição 1):

Passo	j	Comparação	Resultado	Lista depois
1	0	`1 > 2`	Falso: não troca	[**1, 2**, 3, 5, 6, 90]

Fim da varredura 5: o `2` ficou no lugar definitivo. Lista: [1 | 2, 3, 5, 6, 90]



Varredura 5: o `2` terminou no lugar definitivo e a lista ficou ordenada.

Total: 15 comparações, 7 trocas. Resultado: `[1, 2, 3, 5, 6, 90]`

Repare que as varreduras 4 e 5 não trocaram nada: a lista já estava ordenada desde a varredura 3, mas o algoritmo não percebe isso e continua comparando.

3.3 Implementação

```
        def bubble_sort(lista):
            n = len(lista)
            for i in range(0, n - 1):
                for j in range(0, n - 1 - i):
                    if lista[j] > lista[j + 1]:         # compara itens vizinhos
                        aux = lista[j]                  # realiza a troca
                        lista[j] = lista[j + 1]
                        lista[j + 1] = aux
```

Linha a linha:

- `n = len(lista)`: guarda o tamanho da lista.
- `for i in range(0, n - 1)`: cada volta do `i` é **uma varredura**. São `n - 1` varreduras (`i` vale `0, 1, ..., n - 2`).
- `for j in range(0, n - 1 - i)`: `j` é a posição do **primeiro item do par** comparado. Na varredura `i`, os últimos `i` itens já estão no lugar definitivo, então não precisam mais ser comparados. Para `n = 6`:

Varredura (`i`)	`j` vai de... até...	Pares comparados
0	0 até 4	5 pares
1	0 até 3	4 pares
2	0 até 2	3 pares
3	0 até 1	2 pares
4	0 até 0	1 par

- `if lista[j] > lista[j + 1]`: se o item da esquerda é maior que o da direita, eles estão fora de ordem.
- As três linhas seguintes fazem a **troca** vista na seção 2.1.

```
lista = [5, 3, 2, 1, 90, 6]
bubble_sort(lista)
print(lista)                    # [1, 2, 3, 5, 6, 90]
```

3.4 Pontos de atenção

1) Por que `n - 1 - i` e não `n - i`? Dentro do laço usamos `lista[j + 1]`. Se `j` chegar até o último índice, `j + 1` passa do fim da lista. Veja a primeira varredura (`i = 0`) com `n = 6` e o laço `range(0, n)`:

`j`	Comparação	O que acontece
0 a 4	`lista[j] > lista[j + 1]`	funciona normalmente
5	`lista[5] > lista[6]`	`IndexError: list index out of range` (a lista só tem índices de 0 a 5)

O último valor de `j` precisa ser `n - 2`, que dá o par `(n - 2, n - 1)`. Como o `range` não inclui o valor final, escreve-se `range(0, n - 1 - i)`.

2) Por que `>` e não `>=`? Itens **iguais** já estão em ordem entre si. Trocá-los seria trabalho inútil.

3) O bubble sort não percebe que a lista já está ordenada. No exemplo, as varreduras 4 e 5 não trocaram nenhum par e mesmo assim foram feitas. No exercício 4 você vai melhorar isso.

3.5 Desempenho do bubble sort

- **Quantidade de comparações:** na 1ª varredura são `N - 1` comparações, na 2ª `N - 2`, e assim por diante até 1. A soma é `(N - 1) + (N - 2) + ... + 1 = N x (N - 1) / 2`. É um valor **fixo**: não depende de como a lista estava. Para `N = 6`: `6 x 5 / 2 = 15`, exatamente as 15 comparações do quadro acima.
- **Melhor caso:** lista já ordenada. Faz todas as comparações, mas **nenhuma troca**.
- **Pior caso:** lista na **ordem inversa** da desejada. Cada comparação termina em troca, e a cada varredura apenas **um** item chega ao seu lugar definitivo.

A quantidade de operações cresce com o **quadrado** de `N` (`N x N`). Se a lista tem o **dobro** de itens, o trabalho fica **quatro vezes** maior:

Itens na lista (`N`)	Comparações (`N x (N - 1) / 2`)
10	45
100	4.950
1.000	499.500
1.000.000	499.999.500.000 (quase 500 bilhões)

4. Selection sort (ordenação por seleção)

4.1 A ideia

O **selection sort** também é lento e simples, indicado para conjuntos pequenos. A ideia é **selecionar** o item certo para cada posição:

1. encontra o **menor** item da lista e troca com o da **1ª** posição;
2. encontra o **segundo menor** (o menor do que sobrou) e troca com o da **2ª** posição;
3. e assim por diante, até a lista ficar ordenada.

A parte da **esquerda** vai crescendo, ordenada e definitiva; a parte da **direita** é a que ainda falta organizar. Quando só resta um item, ele já é, necessariamente, o maior e está no lugar: por isso são `N - 1` rodadas.

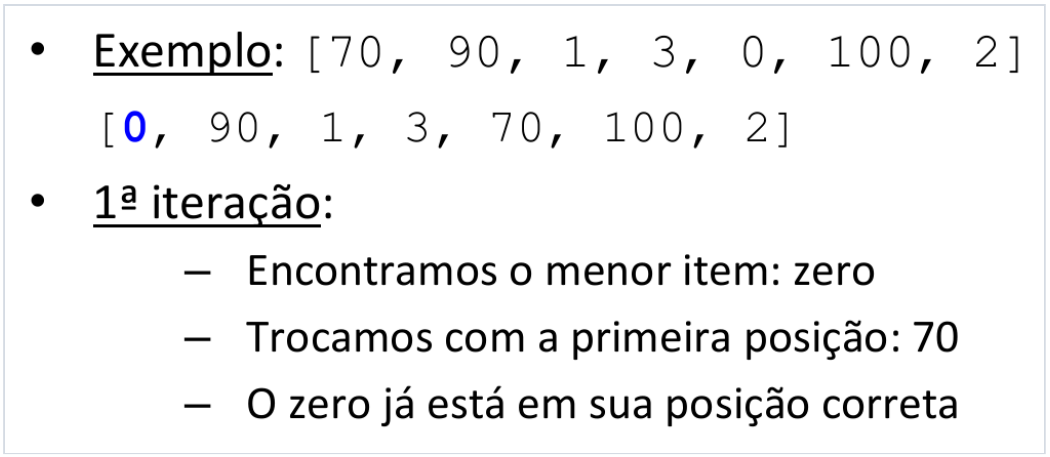
4.2 Exemplo passo a passo

Vamos ordenar `[70, 90, 1, 3, 0, 100, 2]`. Em cada rodada, `menor` guarda a **posição** do menor item visto até agora. Depois de olhar toda a parte que falta, o item da posição `menor` troca de lugar com o da posição `i`. A barra `|` separa a parte já ordenada (à esquerda) da que falta (à direita).

Rodada 1 (`i = 0`): procurar o menor item da posição 0 até a 6. No começo, `menor = 0` (valor `70`).

`j`	Comparação	Resultado	`menor` depois
1	`90 < 70`	Falso: continua	0 (valor `70`)
2	`1 < 70`	Verdadeiro: novo menor	2 (valor `1`)
3	`3 < 1`	Falso: continua	2 (valor `1`)
4	`0 < 1`	Verdadeiro: novo menor	4 (valor `0`)
5	`100 < 0`	Falso: continua	4 (valor `0`)
6	`2 < 0`	Falso: continua	4 (valor `0`)

Troca: `lista[0]` (valor `70`) com `lista[4]` (valor `0`). Lista: [**0** | 90, 1, 3, **70**, 100, 2]

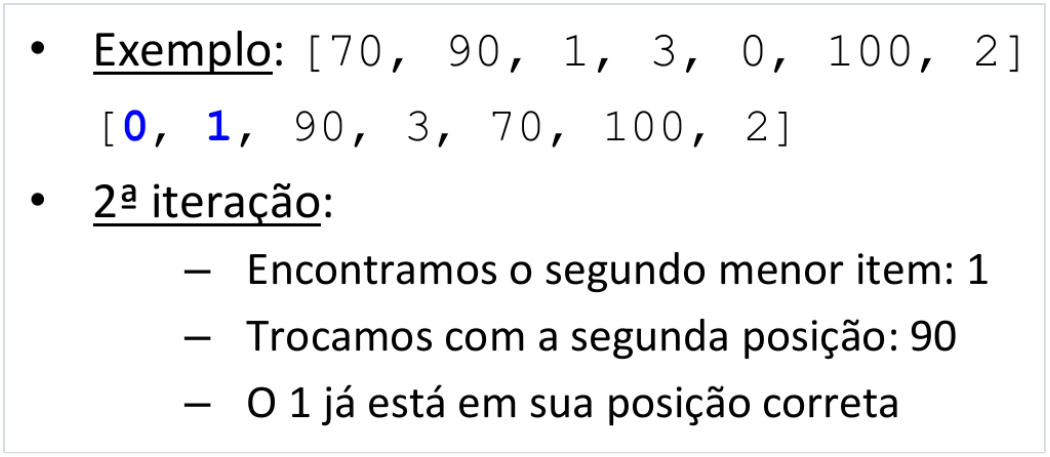


Rodada 1: o menor item (`0`) foi trocado com o `70`.

Rodada 2 (`i = 1`): procurar o menor item da posição 1 até a 6. No começo, `menor = 1` (valor `90`).

j	Comparação	Resultado	`menor` depois
2	`1 < 90`	Verdadeiro: novo menor	2 (valor `1`)
3	`3 < 1`	Falso: continua	2 (valor `1`)
4	`70 < 1`	Falso: continua	2 (valor `1`)
5	`100 < 1`	Falso: continua	2 (valor `1`)
6	`2 < 1`	Falso: continua	2 (valor `1`)

Troca: `lista[1]` (valor `90`) com `lista[2]` (valor `1`). Lista: [0, **1** | **90**, 3, 70, 100, 2]

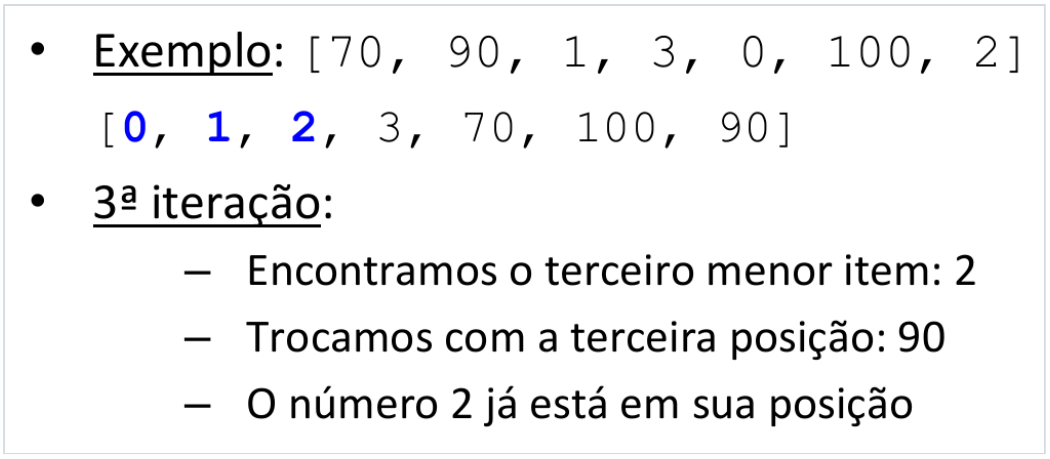


Rodada 2: o menor item restante (`1`) foi trocado com o `90`.

Rodada 3 (`i = 2`): procurar o menor item da posição 2 até a 6. No começo, `menor = 2` (valor `90`).

j	Comparação	Resultado	`menor` depois
3	`3 < 90`	Verdadeiro: novo menor	3 (valor `3`)
4	`70 < 3`	Falso: continua	3 (valor `3`)
5	`100 < 3`	Falso: continua	3 (valor `3`)
6	`2 < 3`	Verdadeiro: novo menor	6 (valor `2`)

Troca: `lista[2]` (valor `90`) com `lista[6]` (valor `2`). Lista: [0, 1, **2** | 3, 70, 100, **90**]

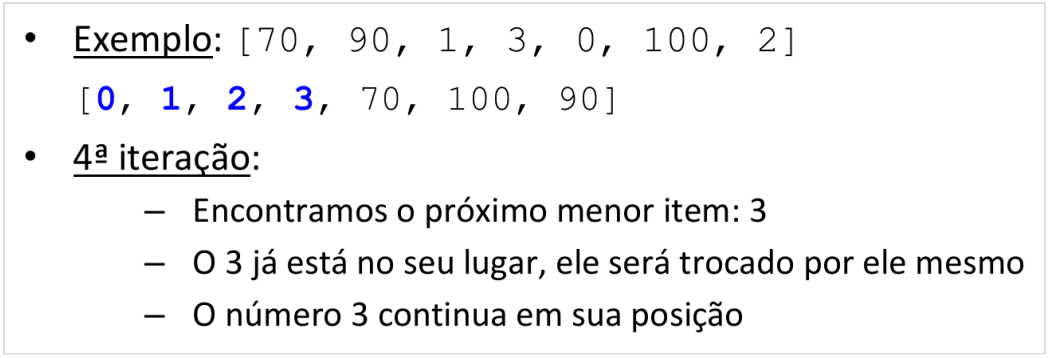


Rodada 3: o menor item restante (`2`) foi trocado com o `90`.

Rodada 4 (`i = 3`): procurar o menor item da posição 3 até a 6. No começo, `menor = 3` (valor `3`).

j	Comparação	Resultado	`menor` depois
4	`70 < 3`	Falso: continua	3 (valor `3`)
5	`100 < 3`	Falso: continua	3 (valor `3`)
6	`90 < 3`	Falso: continua	3 (valor `3`)

Troca: `lista[3]` com `lista[3]` é o **mesmo item** (3): a lista não muda. Lista: [0, 1, 2, 3 | 70, 100, 90]



Rodada 4: o `3` já estava na posição certa (trocou com ele mesmo).

Rodada 5 (`i = 4`): procurar o menor item da posição 4 até a 6. No começo, `menor = 4` (valor `70`).

j	Comparação	Resultado	`menor` depois
5	`100 < 70`	Falso: continua	4 (valor `70`)
6	`90 < 70`	Falso: continua	4 (valor `70`)

Troca: `lista[4]` com `lista[4]` é o **mesmo item** (70): a lista não muda. Lista: [0, 1, 2, 3, 70 | 100, 90]

- Exemplo: `[70, 90, 1, 3, 0, 100, 2]`
  `[0, 1, 2, 3, 70, 100, 90]`
- 5ª iteração:
  - Encontramos o próximo menor item: 70
  - O 70 já está no seu lugar, ele será trocado por ele mesmo
  - O número 70 continua em sua posição

Rodada 5: o `70` já estava na posição certa.

Rodada 6 (`i = 5`): procurar o menor item da posição 5 até a 6. No começo, `menor = 5` (valor `100`).

j	Comparação	Resultado	`menor` depois
6	`90 < 100`	Verdadeiro: novo menor	6 (valor `90`)

Troca: `lista[5]` (valor `100`) com `lista[6]` (valor `90`). Lista: [0, 1, 2, 3, 70, **90** | **100**]

- Exemplo: `[70, 90, 1, 3, 0, 100, 2]`
  `[0, 1, 2, 3, 70, 90, 100]`
- 6ª iteração:
  - Encontramos o próximo menor item: 90
  - O 90 será trocado de posição
  - O número 90 já está em sua posição

Rodada 6: o `90` já estava na posição certa; sobra só o último item, que também já está no lugar.

Total: 21 comparações, 6 trocas. Resultado: `[0, 1, 2, 3, 70, 90, 100]`

Repare nas rodadas 4 e 5: o menor item **já estava** na posição certa, e o código troca o item com **ele mesmo** (a lista não muda). É um pouco de trabalho à toa, que o algoritmo faz de qualquer forma.

4.3 Implementação

```
        def selection_sort(lista):
            n = len(lista)
            for i in range(0, n - 1):
                menor = i
                for j in range(i + 1, n):              # encontra o índice do menor item
                    if lista[j] < lista[menor]:
                        menor = j
                aux = lista[i]                         # realiza a troca
```

```
                lista[i] = lista[menor]
                lista[menor] = aux
```

Linha a linha:

- `for i in range(0, n - 1)`: `i` é a posição que vai receber o próximo menor item (uma rodada por valor de `i`).
- `menor = i`: começamos supondo que o menor item é o da própria posição `i`. A variável guarda o **índice** do menor, e não o valor.
- `for j in range(i + 1, n)`: percorre só o que **falta** (da posição seguinte até o fim).
- `if lista[j] < lista[menor]`: achou um item menor que o menor visto até agora? Então `menor = j`.
- As três últimas linhas (**fora** do laço do `j`) trocam o item da posição `i` com o da posição `menor`. É uma única troca por rodada.

```
    lista = [70, 90, 1, 3, 0, 100, 2]
    selection_sort(lista)
    print(lista)                    # [0, 1, 2, 3, 70, 90, 100]
```

4.4 Pontos de atenção

1) `menor` guarda o índice, não o valor. Para trocar, é preciso saber **onde** o menor item está. Se `menor` guardasse só o valor (`70`, depois `1`, depois `0`...), no fim da rodada saberíamos qual é o menor valor, mas não teríamos como saber de qual posição ele veio.

2) A troca fica depois do laço interno. Uma troca por rodada basta. A troca só pode acontecer quando já olhamos **todos** os itens que faltam, porque só então sabemos qual é o menor.

3) Troca com ele mesmo. Quando o menor já está na posição `i`, a troca não muda nada. Poderia ser evitada com `if menor != i:`, mas a versão da aula não faz essa verificação (e, nos exercícios, essas trocas contam).

4.5 Desempenho do selection sort

- **Comparações:** para achar o menor item, a rodada precisa olhar **todos** os itens que faltam, não importa como estejam: `(N - 1) + (N - 2) + ... + 1 = N x (N - 1) / 2`, **sempre**. Para `N = 7`, `7 x 6 / 2 = 21`: as 21 comparações do quadro.
- **Trocas:** uma por rodada, no máximo `N - 1` (6 no exemplo). É **muito menos** que o bubble sort, que pode chegar a `N x (N - 1) / 2` trocas. Essa é a vantagem do selection sort.
- **Desvantagem:** se a lista já estiver ordenada, ele não ganha nada: o número de comparações continua o mesmo.

5. Insertion sort (ordenação por inserção)

5.1 A ideia

O **insertion sort** também é simples e indicado para conjuntos pequenos. É a forma como muita gente organiza as cartas na mão: pega a carta seguinte, compara com as que já estão em ordem à esquerda e a "desliza" para o lugar certo.

1. começa pelo **2º item** (o 1º, sozinho, já forma uma parte ordenada);
2. compara o item com o da sua **esquerda**; se ele for **menor**, os dois trocam de lugar;
3. repete a comparação com o novo vizinho da esquerda, e assim por diante;
4. **para** quando o vizinho da esquerda não for maior ou quando o item chegar à **1ª posição** da lista;
5. passa para o próximo item.

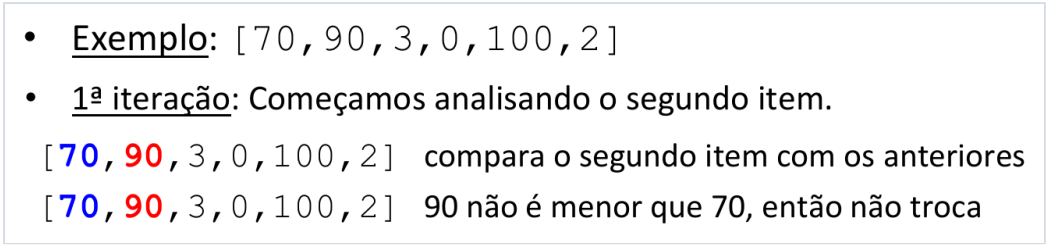
A parte da esquerda fica sempre ordenada, e a cada rodada ela cresce um item. (Diferente do selection sort, os itens da esquerda ainda não são definitivos: um item que chega depois pode empurrá-los para a direita.)

5.2 Exemplo passo a passo

Vamos ordenar `[70, 90, 3, 0, 100, 2]`. Em cada rodada, o item da posição `i` é "inserido" na parte da esquerda. O par comparado aparece em **negrito**.

Rodada 1 (`i = 1`): inserir o item `90` na parte da esquerda ([70], já ordenada).

Passo	j	Comparação	Resultado	Lista depois
1	1	`90 < 70`	Falso: para	[**70, 90**, 3, 0, 100, 2]



Rodada 1: o `90` não é menor que o `70`, então não troca.

Rodada 2 (`i = 2`): inserir o item `3` na parte da esquerda ([70, 90], já ordenada).

Passo	j	Comparação	Resultado	Lista depois
1	2	`3 < 90`	Verdadeiro: **troca**	[70, **3, 90**, 0, 100, 2]
2	1	`3 < 70`	Verdadeiro: **troca**	[**3, 70**, 90, 0, 100, 2]
	0	(chegou à posição 0)	Para: não há item à esquerda	[**3**, 70, 90, 0, 100, 2]

- Exemplo: `[70,90,3,0,100,2]`
- 2ª iteração: Agora analisamos o terceiro item.

`[70,90,3,0,100,2]`
`[70,3,90,0,100,2]`
`[3,70,90,0,100,2]` 3 é menor que 70, então troca

Rodada 2: o `3` é menor que o `90` e depois que o `70`: foi trocado duas vezes e chegou ao começo.

Rodada 3 (`i = 3`): inserir o item `0` na parte da esquerda ([3, 70, 90], já ordenada).

Passo	j	Comparação	Resultado	Lista depois
1	3	`0 < 90`	Verdadeiro: **troca**	[3, 70, **0, 90**, 100, 2]
2	2	`0 < 70`	Verdadeiro: **troca**	[3, **0, 70**, 90, 100, 2]
3	1	`0 < 3`	Verdadeiro: **troca**	[**0, 3**, 70, 90, 100, 2]
	0	(chegou à posição 0)	Para: não há item à esquerda	[**0**, 3, 70, 90, 100, 2]

- Exemplo: `[70,90,3,0,100,2]`
- 3ª iteração: Agora analisamos o quarto item.

`[3,70,90,0,100,2]`
`[3,70,0,90,100,2]`
`[3,0,70,90,100,2]`
`[0,3,70,90,100,2]` zero é menor que 3, então troca

Rodada 3: o `0` foi trocado três vezes e chegou ao começo.

Rodada 4 (`i = 4`): inserir o item `100` na parte da esquerda ([0, 3, 70, 90], já ordenada).

Passo	j	Comparação	Resultado	Lista depois
1	4	`100 < 90`	Falso: para	[0, 3, 70, **90, 100**, 2]

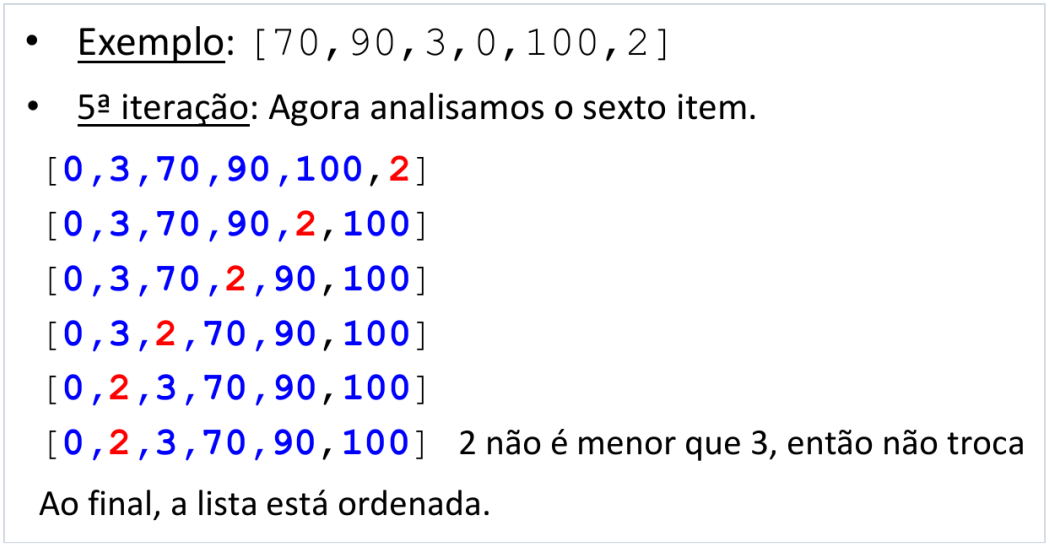
- Exemplo: `[70,90,3,0,100,2]`
- 4ª iteração: Agora analisamos o quinto item.

`[0,3,70,90,100,2]` compara o quinto item com os anteriores
`[0,3,70,90,100,2]` 100 não é menor que 90, então não troca

Rodada 4: o `100` não é menor que o `90`, então não troca.

Rodada 5 (`i = 5`): inserir o item `2` na parte da esquerda ([0, 3, 70, 90, 100], já ordenada).

Passo	j	Comparação	Resultado	Lista depois
1	5	`2 < 100`	Verdadeiro: **troca**	[0, 3, 70, 90, **2, 100**]
2	4	`2 < 90`	Verdadeiro: **troca**	[0, 3, 70, **2, 90**, 100]
3	3	`2 < 70`	Verdadeiro: **troca**	[0, 3, **2, 70**, 90, 100]
4	2	`2 < 3`	Verdadeiro: **troca**	[0, **2, 3**, 70, 90, 100]
5	1	`2 < 0`	Falso: para	[**0, 2**, 3, 70, 90, 100]



Rodada 5: o `2` foi trocado quatro vezes, até ficar depois do `0`; a lista ficou ordenada.

Total: 12 comparações, 9 trocas. Resultado: `[0, 2, 3, 70, 90, 100]`

5.3 Implementação

```
        def insertion_sort(lista):
            n = len(lista)
            for i in range(1, n):
                j = i
                while j > 0 and lista[j] < lista[j - 1]:    # compara com o item da esquerda
                    aux = lista[j]                          # realiza a troca
                    lista[j] = lista[j - 1]
                    lista[j - 1] = aux
                    j -= 1
```

Linha a linha:

- `for i in range(1, n)`: `i` é a posição do item que será inserido. Começa em `1` (o 2º item).
- `j = i`: `j` é a posição **atual** do item que está andando para a esquerda. Ele vai diminuindo a cada troca.
- `while j > 0 and lista[j] < lista[j - 1]`: continua **enquanto** o item não chegou à posição 0 **e** ainda é menor que o vizinho da esquerda.
- As três linhas de troca levam o item uma casa para a esquerda.
- `j -= 1`: o item agora está uma posição mais à esquerda, então `j` diminui.

```
lista = [70, 90, 3, 0, 100, 2]
insertion_sort(lista)
print(lista)                    # [0, 2, 3, 70, 90, 100]
```

5.4 Pontos de atenção

1) A ordem das condições do `while` importa. O `and` avalia da esquerda para a direita e **para no primeiro falso**. Por isso `j > 0` vem primeiro: quando `j` vale `0`, o `while` já para ali, sem olhar `lista[j - 1]`. Se as condições fossem invertidas (`lista[j] < lista[j - 1] and j > 0`), com `j = 0` o Python compararia `lista[0]` com `lista[-1]` (o **último** item da lista, lembra do índice negativo da Aula 13?). O resultado final não mudaria, mas faria uma comparação inútil, e em outras linguagens esse acesso seria um erro.

2) Por que `while` e não `for`? Não dá para saber de antemão quantas casas o item vai andar: depende do valor dele. No exemplo, o `90` andou zero casas e o `2` andou quatro. O `while` repete "enquanto for preciso".

3) Esquecer o `j -= 1`. Sem ele, o item só andaria **uma** casa por rodada, e o resultado ficaria errado. Veja `[3, 2, 1]` sem o `j -= 1`:

Rodada	`i`	`j`	Comparação	O que acontece
1	1	1	`2 < 3`	Troca: `[2, 3, 1]`
2	2	2	`1 < 3`	Troca: `[2, 1, 3]`. O `j` continua 2, e agora `lista[2]` (3) `<` `lista[1]` (1) é falso: **para**, com o `1` fora do lugar

A lista termina como `[2, 1, 3]`, que **não** está ordenada.

5.5 Desempenho do insertion sort

- **Melhor caso:** lista já ordenada. Cada item é comparado **uma única vez** com o vizinho da esquerda, descobre que não é menor e para. São `N - 1` comparações e **nenhuma** troca.
- **Pior caso:** lista na ordem **inversa**. Cada item precisa andar até a primeira posição: o item da posição `i` faz `i` comparações e `i` trocas. A soma é `1 + 2 + ... + (N - 1) = N x (N - 1) / 2` comparações e também `N x (N - 1) / 2` trocas.
- **Caso médio** (lista embaralhada): cada item anda, em média, metade do caminho, então o trabalho é cerca de **metade** do pior caso.

Por isso o insertion sort faz **menos comparações** que os outros dois métodos simples: ele só vai até o início da lista no pior caso. Nos outros casos, para antes. Ele também é o que mais aproveita uma lista que já está **quase ordenada**.

6. Comparação dos métodos simples

Resumo das operações para uma lista de `N` itens (as comparações do insertion sort contam todas as vezes em que `lista[j] < lista[j - 1]` é avaliado):

Método	Comparações (melhor caso)	Comparações (pior caso)	Trocas (melhor caso)	Trocas (pior caso)
Bubble sort	`N x (N - 1) / 2`	`N x (N - 1) / 2`	0	`N x (N - 1) / 2`
Selection sort	`N x (N - 1) / 2`	`N x (N - 1) / 2`	`N - 1`	`N - 1`
Insertion sort	`N - 1`	`N x (N - 1) / 2`	0	`N x (N - 1) / 2`

Os exercícios 2 e 3 pedem que você **confira** essa tabela na prática, para uma lista de 1.000 itens (prever os números pelas fórmulas **antes** de executar o programa é um bom treino).

E em uma lista **embaralhada**, que é o caso mais comum? Veja as operações para uma lista aleatória de 2.000 itens (a mesma lista do exercício 7):

Método	Comparações	Trocas	Comparações + trocas
Bubble sort	1.999.000	987.600	2.986.600
Selection sort	1.999.000	1.999	2.000.999
Insertion sort	989.593	987.600	1.977.193

O que se vê:

- o **insertion sort** faz cerca de **metade** das comparações dos outros dois;
- o **selection sort** faz pouquíssimas **trocas** (menos de 2 mil, contra quase 1 milhão dos outros);
- somando as duas operações, insertion e selection ficam **próximos**, e o **bubble sort** é o que mais trabalha. Qual dos dois é mais rápido de fato depende do computador e da lista (veja as medições de tempo mais adiante).

Em todos os três, o trabalho cresce com o **quadrado** do tamanho da lista. Por isso eles servem para listas pequenas.

7. Métodos sofisticados: divisão e conquista

Os dois métodos sofisticados desta aula, **merge sort** e **quick sort**, usam a técnica de **divisão e conquista** que vimos na Aula 12: dividir um problema grande em problemas menores, resolver cada um (conquistar) e combinar as respostas. São, portanto, algoritmos **recursivos**.

Por que isso ajuda? Ordenar uma lista pequena é bem mais barato que ordenar uma grande, porque o custo dos métodos simples cresce com `N x N`. Ao partir a lista em pedaços, os algoritmos evitam comparar cada item com quase todos os outros. O custo dos métodos sofisticados cresce com `N x log2(N)` (o `log2` é o mesmo "quantas vezes dá para dividir por 2" da busca binária):

Itens (`N`)	`N x N` (métodos simples)	`N x log2(N)` (métodos sofisticados)
10	100	cerca de 33
1.000	1.000.000	cerca de 10 mil
1.000.000	1.000.000.000.000 (1 trilhão)	cerca de 20 milhões

Para 1 milhão de itens, a diferença é de cerca de **50 mil vezes**.

8. Merge sort (ordenação por intercalação)

8.1 A ideia

O **merge sort** tem duas etapas:

- **Dividir:** a lista de `N` itens é dividida em duas metades (de tamanhos aproximadamente iguais). Cada metade é dividida de novo, e assim sucessivamente, até sobrarem listas de **um único item**. Uma lista de um item já está ordenada.
- **Conquistar:** voltando, as listas ordenadas são **intercaladas** duas a duas, formando listas ordenadas cada vez maiores, até restar uma só.

Intercalar duas listas **já ordenadas** é fácil: só é preciso olhar o **primeiro item de cada uma**. O menor dos dois é o menor de todos, e vai para a lista de resultado. Repete-se até uma delas acabar, e então copia-se o que sobrou da outra.

8.2 Exemplo passo a passo

Vamos ordenar `[5, 4, 1, 0, 7, 6, 3, 2]` (8 itens). Primeiro, só a **ideia geral**, em níveis. Dividindo (de cima para baixo):

```
[5, 4, 1, 0, 7, 6, 3, 2]
[5, 4, 1, 0]              [7, 6, 3, 2]
[5, 4]      [1, 0]        [7, 6]      [3, 2]
[5]   [4]   [1]   [0]     [7]   [6]    [3]   [2]
```

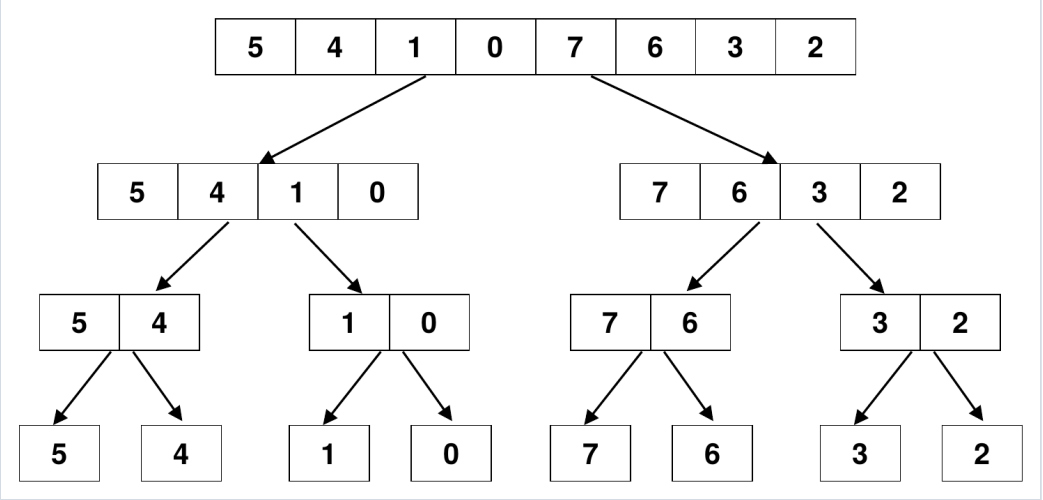
Intercalando (de baixo para cima):

```
[5]   [4]   [1]   [0]     [7]   [6]    [3]   [2]
[4, 5]      [0, 1]        [6, 7]      [2, 3]
[0, 1, 4, 5]              [2, 3, 6, 7]
[0, 1, 2, 3, 4, 5, 6, 7]
```

São 3 níveis de divisão: `log2(8) = 3`. Agora, a **ordem real** em que o programa executa, que é diferente da imagem por níveis. Como a função é recursiva, ela desce por completo na primeira metade (e a ordena) antes de olhar a segunda.

Fase 1 (descendo): as chamadas de `merge_sort`. A coluna `Nível` mostra a profundidade da recursão (0 é a primeira chamada).

Passo	Nível	Chamada	Parte da lista analisada	O que acontece
1	0	`merge_sort(lista, 0, 7)`	`[5, 4, 1, 0, 7, 6, 3, 2]`	divide em `0..3` e `4..7`
2	1	`merge_sort(lista, 0, 3)`	`[5, 4, 1, 0]`	divide em `0..1` e `2..3`
3	2	`merge_sort(lista, 0, 1)`	`[5, 4]`	divide em `0..0` e `1..1`
4	3	`merge_sort(lista, 0, 0)`	`[5]`	`inicio == fim`: um único item, já está ordenado. Volta.
5	3	`merge_sort(lista, 1, 1)`	`[4]`	`inicio == fim`: um único item, já está ordenado. Volta.
6	2	`merge_sort(lista, 2, 3)`	`[1, 0]`	divide em `2..2` e `3..3`
7	3	`merge_sort(lista, 2, 2)`	`[1]`	`inicio == fim`: um único item, já está ordenado. Volta.
8	3	`merge_sort(lista, 3, 3)`	`[0]`	`inicio == fim`: um único item, já está ordenado. Volta.
9	1	`merge_sort(lista, 4, 7)`	`[7, 6, 3, 2]`	divide em `4..5` e `6..7`
10	2	`merge_sort(lista, 4, 5)`	`[7, 6]`	divide em `4..4` e `5..5`
11	3	`merge_sort(lista, 4, 4)`	`[7]`	`inicio == fim`: um único item, já está ordenado. Volta.
12	3	`merge_sort(lista, 5, 5)`	`[6]`	`inicio == fim`: um único item, já está ordenado. Volta.
13	2	`merge_sort(lista, 6, 7)`	`[3, 2]`	divide em `6..6` e `7..7`
14	3	`merge_sort(lista, 6, 6)`	`[3]`	`inicio == fim`: um único item, já está ordenado. Volta.
15	3	`merge_sort(lista, 7, 7)`	`[2]`	`inicio == fim`: um único item, já está ordenado. Volta.

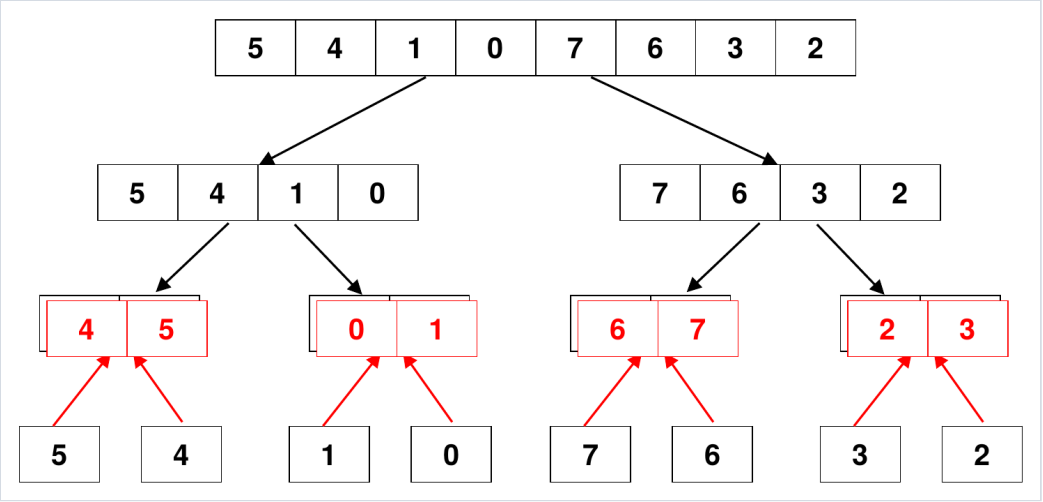


Fim da Fase 1: a lista foi dividida até sobrar um item em cada pedaço.

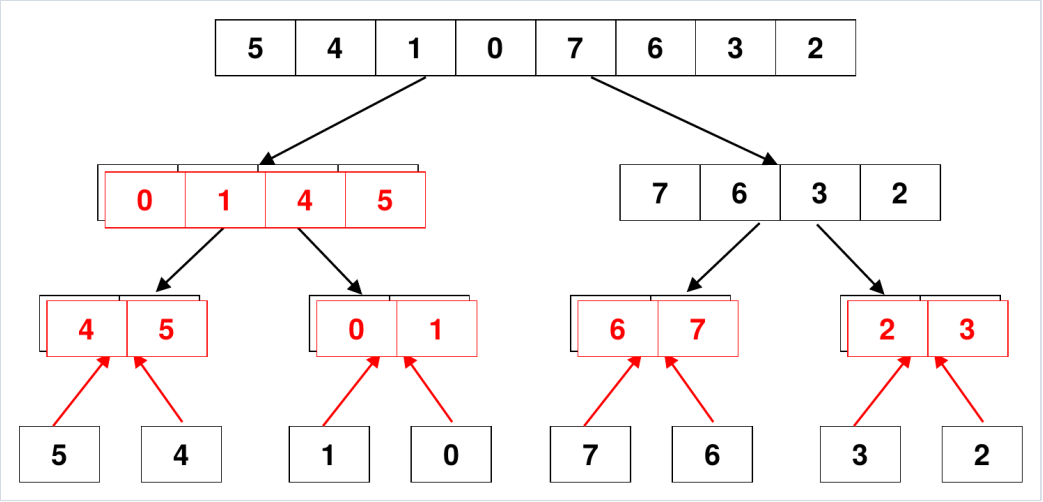
Ao chegar em um único item (`inicio == fim`), a função não faz nada e volta. É a **condição de parada** da recursão (Aula 12).

Fase 2 (voltando): as intercalações. Cada vez que as duas chamadas de baixo terminam, a função `intercala` junta as duas metades:

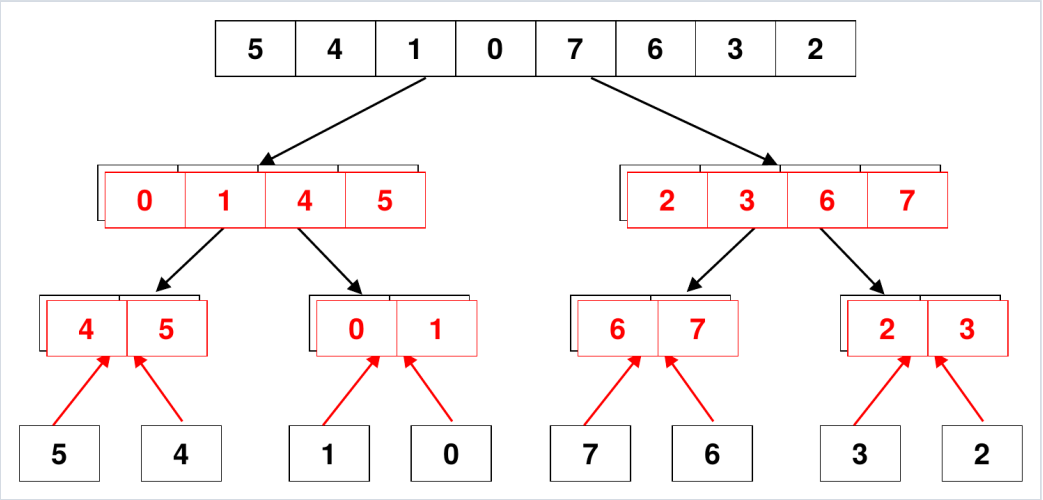
Passo	Chamada	Sublista esquerda	Sublista direita	Resultado (`aux`)	Lista inteira depois
1	`intercala(lista, 0, 0, 1)`	`[5]`	`[4]`	`[4, 5]`	`[4, 5, 1, 0, 7, 6, 3, 2]`
2	`intercala(lista, 2, 2, 3)`	`[1]`	`[0]`	`[0, 1]`	`[4, 5, 0, 1, 7, 6, 3, 2]`
3	`intercala(lista, 0, 1, 3)`	`[4, 5]`	`[0, 1]`	`[0, 1, 4, 5]`	`[0, 1, 4, 5, 7, 6, 3, 2]`
4	`intercala(lista, 4, 4, 5)`	`[7]`	`[6]`	`[6, 7]`	`[0, 1, 4, 5, 6, 7, 3, 2]`
5	`intercala(lista, 6, 6, 7)`	`[3]`	`[2]`	`[2, 3]`	`[0, 1, 4, 5, 6, 7, 2, 3]`
6	`intercala(lista, 4, 5, 7)`	`[6, 7]`	`[2, 3]`	`[2, 3, 6, 7]`	`[0, 1, 4, 5, 2, 3, 6, 7]`
7	`intercala(lista, 0, 3, 7)`	`[0, 1, 4, 5]`	`[2, 3, 6, 7]`	`[0, 1, 2, 3, 4, 5, 6, 7]`	`[0, 1, 2, 3, 4, 5, 6, 7]`



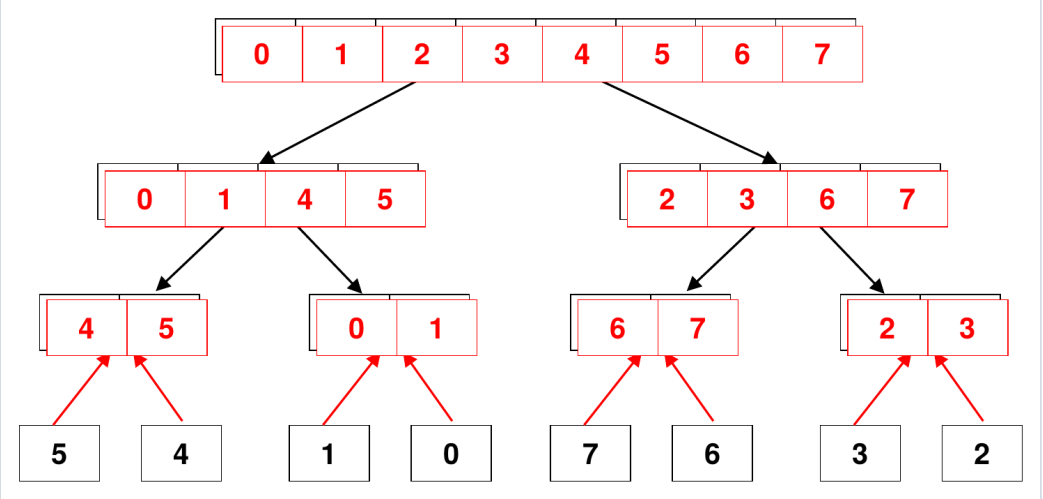
Fase 2, começo: os itens soltos foram intercalados dois a dois (`[4, 5]`, `[0, 1]`, `[6, 7]`, `[2, 3]`).



As duas primeiras sublistas foram intercaladas, formando `[0, 1, 4, 5]`.



Também a metade da direita foi intercalada, formando `[2, 3, 6, 7]`.



A última intercalação junta as duas metades e produz a lista ordenada.

Repare que as intercalações 1 a 3 ordenam a **primeira metade** da lista por completo, antes de a segunda metade ser mexida.

Veja agora **por dentro** de duas intercalações. A primeira é a mais simples, com um item de cada lado. `esq` é a posição atual na sublista da esquerda, e `dir`, na da direita:

Passo	`esq`	`dir`	Comparação	Resultado	O que acontece	`aux` depois
1	0	1	`5 <= 4`	Falso	copia 4 (da direita)	`[4]`
2	0	-	(direita acabou)		copia 5 (resto da esquerda)	`[4, 5]`

Fim: `aux = [4, 5]` é copiada de volta para as posições 0 a 1.

E a última, que junta as duas metades já ordenadas:

Passo	`esq`	`dir`	Comparação	Resultado	O que acontece	`aux` depois
1	0	4	`0 <= 2`	Verdadeiro	copia 0 (da esquerda)	`[0]`
2	1	4	`1 <= 2`	Verdadeiro	copia 1 (da esquerda)	`[0, 1]`
3	2	4	`4 <= 2`	Falso	copia 2 (da direita)	`[0, 1, 2]`
4	2	5	`4 <= 3`	Falso	copia 3 (da direita)	`[0, 1, 2, 3]`
5	2	6	`4 <= 6`	Verdadeiro	copia 4 (da esquerda)	`[0, 1, 2, 3, 4]`
6	3	6	`5 <= 6`	Verdadeiro	copia 5 (da esquerda)	`[0, 1, 2, 3, 4, 5]`
7	-	6	(esquerda acabou)		copia 6 (resto da direita)	`[0, 1, 2, 3, 4, 5, 6]`
8	-	7	(esquerda acabou)		copia 7 (resto da direita)	`[0, 1, 2, 3, 4, 5, 6, 7]`

Fim: `aux = [0, 1, 2, 3, 4, 5, 6, 7]` é copiada de volta para as posições 0 a 7.

Nos passos 7 e 8, a sublista da esquerda já acabou. Como o resto da direita já está ordenado, ele é simplesmente copiado.

8.3 Implementação

O merge sort usa duas funções: `merge_sort`, que divide a lista recursivamente, e `intercala`, que junta duas sublistas ordenadas.

```
        def intercala(lista, inicio, meio, fim):
            aux = []                                    # lista auxiliar
            esq = inicio
            dir = meio + 1
            while esq <= meio and dir <= fim:           # enquanto nenhuma sublista acabou,
                if lista[esq] <= lista[dir]:            # copia o menor item para aux
                    aux.append(lista[esq])
                    esq += 1
                else:
                    aux.append(lista[dir])
                    dir += 1
            while esq <= meio:                          # copia o resto da primeira sublista
                aux.append(lista[esq])
                esq += 1
```

```
        while dir <= fim:                           # copia o resto da segunda sublista
            aux.append(lista[dir])
            dir += 1
        lista[inicio:fim + 1] = aux                 # copia aux de volta para a lista
        return lista


    def merge_sort(lista, inicio, fim):
        meio = (fim + inicio) // 2
        if inicio < fim:
            merge_sort(lista, inicio, meio)         # primeira metade da lista
            merge_sort(lista, meio + 1, fim)        # segunda metade da lista
            intercala(lista, inicio, meio, fim)     # intercala as duas metades
```

Linha a linha, em `merge_sort`:

- `inicio` e `fim` são os índices que delimitam a parte da lista que a chamada deve ordenar (como na busca binária recursiva da Aula 13). Para ordenar a lista toda: `merge_sort(lista, 0, len(lista) - 1)`.
- `meio = (fim + inicio) // 2`: posição do meio da parte atual.
- `if inicio < fim`: só divide se a parte tem **pelo menos 2 itens**. Com 1 item (`inicio == fim`) não há o que fazer: é a condição de parada.
- As duas chamadas recursivas ordenam cada metade; depois, `intercala` junta as duas.

Linha a linha, em `intercala`:

- A **primeira sublista** vai de `inicio` até `meio`, e a **segunda**, de `meio + 1` até `fim`. As duas já estão ordenadas.
- `esq` e `dir` são as posições atuais em cada sublista. `aux` é uma lista nova que recebe os itens na ordem certa.
- O primeiro `while` roda enquanto **as duas** sublistas ainda têm itens: copia para `aux` o menor dos dois itens da vez e avança na sublista de onde ele saiu.
- Os dois últimos `while` copiam o que sobrou. **Só um** deles chega a executar: o primeiro `while` terminou porque uma sublista acabou.
- `lista[inicio:fim + 1] = aux`: copia `aux` de volta para as posições `inicio` a `fim` da lista original. O `+ 1` existe porque o fim de uma fatia não é incluído.

```
    lista = [5, 4, 1, 0, 7, 6, 3, 2]
    merge_sort(lista, 0, len(lista) - 1)
    print(lista)                    # [0, 1, 2, 3, 4, 5, 6, 7]
```

8.4 Pontos de atenção

1) `<=` e não `<` na intercalação. Com valores **iguais**, o `<=` copia primeiro o item da **esquerda**, que já vinha antes na lista. Assim, itens iguais mantêm a ordem original entre si. Essa propriedade se chama **estabilidade**, e é importante quando se ordena registros por um

campo (como produtos por preço): produtos de mesmo preço continuam na ordem em que estavam. O merge sort é **estável**.

2) O nome `dir`. Em Python, `dir` já é o nome de uma função pronta. Usá-lo como variável local, como no código da aula, funciona (só vale dentro de `intercala`), mas esconde a função original. É melhor evitar nomes de funções do Python em variáveis.

3) Lista auxiliar. A intercalação cria a lista `aux`, o que usa **memória extra**, proporcional ao tamanho da lista. Os métodos simples e o quick sort ordenam no próprio lugar, sem essa lista.

4) A lista vazia funciona. Com `fim = -1` (e `inicio = 0`), a condição `inicio < fim` é falsa e nada acontece.

5) Profundidade da recursão. A lista é dividida **ao meio** a cada chamada, então a pilha cresce só `log2(N)` níveis (cerca de 20 para 1 milhão de itens). Não há risco de `RecursionError` (Aula 12).

8.5 Desempenho do merge sort

- **Não depende da ordem inicial:** lista ordenada, inversa ou embaralhada fazem praticamente o mesmo trabalho. Para o merge sort, melhor e pior caso são parecidos.
- **Quantidade de operações:** a lista é dividida em `log2(N)` níveis, e em cada nível a intercalação percorre, no máximo, `N` itens. Total da ordem de `N x log2(N)`. Com 8 itens são 3 níveis, e no máximo 8 comparações em cada um (o exemplo acima fez 14 comparações no total, bem menos que 24).
- **Custo:** a memória extra da lista `aux`.

9. Quick sort (ordenação rápida)

9.1 A ideia

O **quick sort** também usa divisão e conquista, mas de outro jeito: a operação central é o **particionamento**.

O particionamento escolhe um item da lista, chamado de **pivô**, e o coloca em uma posição tal que:

- todos os itens à **esquerda** dele são **menores ou iguais** a ele;
- todos os itens à **direita** são **maiores**.

Depois disso, o pivô **já está no lugar definitivo** (a posição dele é a que teria na lista ordenada). A lista **não** está ordenada ainda: as duas partes (esquerda e direita) estão apenas separadas. O mesmo processo é repetido, recursivamente, para cada parte.

9.2 O particionamento passo a passo

O pivô pode ser o primeiro item, o último ou um item **sorteado**. Para começar a entender, vamos usar o **primeiro item** como pivô, com a lista `[4, 6, 2, 5, 3, 9]`. O algoritmo usa duas variáveis:

- `i`: a posição do **fim da região dos menores ou iguais** ao pivô. Começa na posição do pivô;
- `j`: percorre a lista, da 2ª posição até o fim, olhando um item por vez.

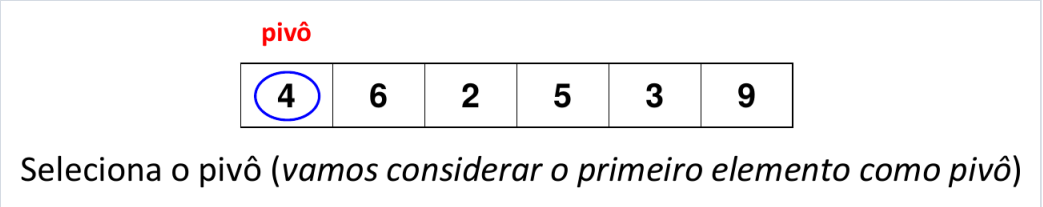
Para cada item (`j`): se ele for **menor ou igual** ao pivô, `i` avança uma posição e o item da posição `j` troca de lugar com o da posição `i` (ou seja, ele é levado para a região dos menores). Se for **maior**, nada acontece. No fim, o pivô troca de lugar com o item da posição `i`, e fica entre as duas regiões.

Pivô = primeiro item = `4`. `i = 0` (posição do pivô). A lista é percorrida de `j = 1` até `5`.

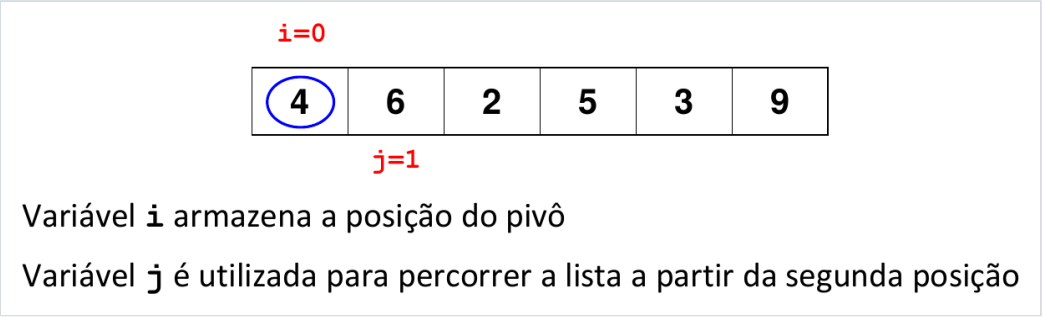
`j`	`lista[j]`	Comparação	`i` depois	Lista depois
1	6	`6 <= 4`: Falso	0	[4, **6**, 2, 5, 3, 9] (não faz nada)
2	2	`2 <= 4`: Verdadeiro	1	[4, **2, 6**, 5, 3, 9] (troca)
3	5	`5 <= 4`: Falso	1	[4, 2, 6, **5**, 3, 9] (não faz nada)
4	3	`3 <= 4`: Verdadeiro	2	[4, 2, **3**, 5, **6**, 9] (troca)
5	9	`9 <= 4`: Falso	2	[4, 2, 3, 5, 6, **9**] (não faz nada)

Último passo: troca o pivô (posição 0) com a posição `i = 2`: [**3**, 2, **4**, 5, 6, 9]

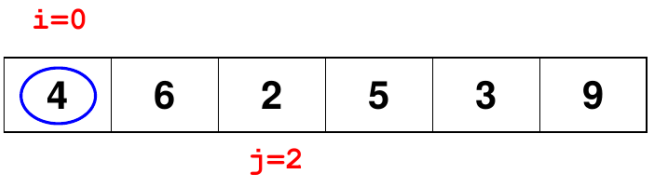
O pivô ficou na posição 2. Menores ou iguais: `[3, 2]`. Maiores: `[5, 6, 9]`.



O primeiro item (`4`) é escolhido como pivô.



`i` marca a posição do pivô (0) e `j` começa na segunda posição (1).

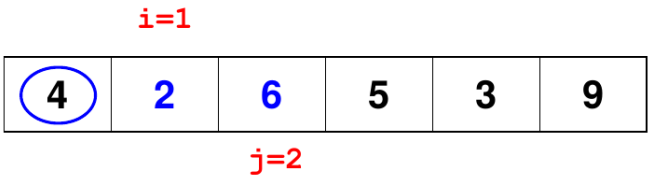


Comparar elemento da posição **j** com o valor do pivô

Se o elemento for menor ou igual que o pivô:

- incrementa valor da variável **i**
- troca elemento da posição **i** com o elemento da posição **j**

`j = 2`: o `2` é comparado com o pivô `4`.

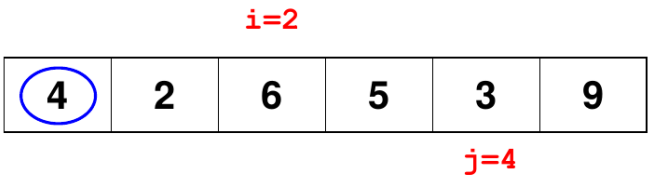


Comparar elemento da posição **j** com o valor do pivô

Se o elemento for menor ou igual que o pivô:

- incrementa valor da variável **i**
- troca elemento da posição **i** com o elemento da posição **j**

O `2` é menor que o pivô: `i` avança para 1 e o `2` troca de lugar com o `6`.

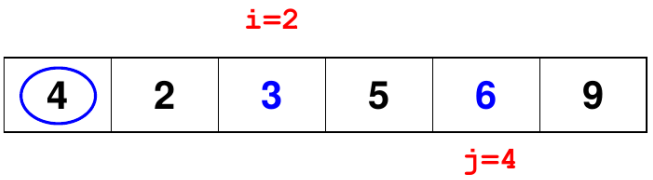


Comparar elemento da posição **j** com o valor do pivô

Se o elemento for menor ou igual que o pivô:

- incrementa valor da variável **i**
- troca elemento da posição **i** com o elemento da posição **j**

`j = 4`: o `3` é menor que o pivô, então `i` avança para 2 e o `3` vai trocar com o `6`.

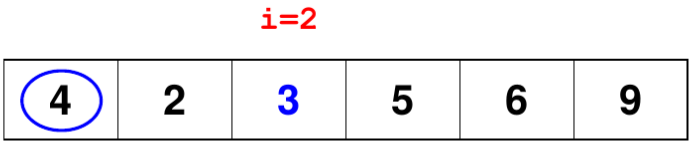


Comparar elemento da posição **`j`** com o valor do pivô

Se o elemento for menor ou igual que o pivô:

- incrementa valor da variável **`i`**
- troca elemento da posição **`i`** com o elemento da posição **`j`**

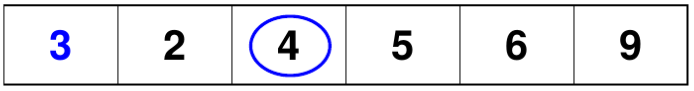
Depois da troca: `[4, 2, 3, 5, 6, 9]`.



Após percorrer a lista, executamos um último passo:

- Trocar de posição o elemento da posição **`i`** com o pivô

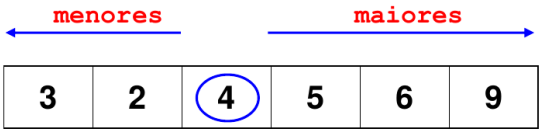
O `j` chegou ao fim da lista. Falta o último passo: trocar o pivô com a posição `i`.



Após percorrer a lista, executamos um último passo:

- Trocar de posição o elemento da posição **`i`** com o pivô

O pivô (`4`) trocou com o item da posição `i = 2`: `[3, 2, 4, 5, 6, 9]`.



Ao final, teremos todos os elementos menores que o pivô à esquerda, e todos os maiores à direita

Observe que a lista ainda não está ordenada. Temos duas sublistas (direita e esquerda).

Esse processo de particionamento se repetirá recursivamente para cada sublista

Resultado: menores à esquerda, maiores à direita e o pivô no lugar definitivo.

O código da aula **sorteia** o pivô. Para continuar usando o mesmo algoritmo, ele leva o pivô sorteado para a primeira posição (trocando-o com o primeiro item) e então faz exatamente o que acabamos de ver. Por exemplo, com a mesma lista, se o sorteio escolher o índice `3` (o item `5`):

Primeiro, o pivô troca de lugar com o primeiro item: [**5**, 6, 2, **4**, 3, 9]

Pivô = primeiro item = `5`. `i = 0` (posição do pivô). A lista é percorrida de `j = 1` até `5`.

`j`	`lista[j]`	Comparação	`i` depois	Lista depois
1	6	`6 <= 5`: Falso	0	[5, **6**, 2, 4, 3, 9] (não faz nada)
2	2	`2 <= 5`: Verdadeiro	1	[5, **2, 6**, 4, 3, 9] (troca)
3	4	`4 <= 5`: Verdadeiro	2	[5, 2, **4, 6**, 3, 9] (troca)
4	3	`3 <= 5`: Verdadeiro	3	[5, 2, 4, **3, 6**, 9] (troca)
5	9	`9 <= 5`: Falso	3	[5, 2, 4, 3, 6, **9**] (não faz nada)

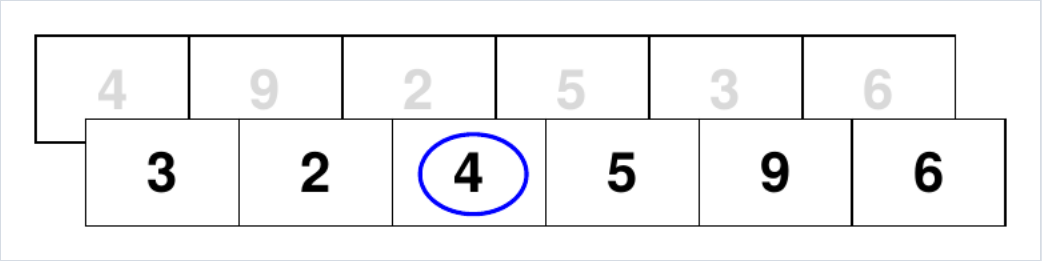
Último passo: troca o pivô (posição 0) com a posição `i = 3`: [**3**, 2, 4, **5**, 6, 9]

O pivô ficou na posição 3. Menores ou iguais: `[3, 2, 4]`. Maiores: `[6, 9]`.

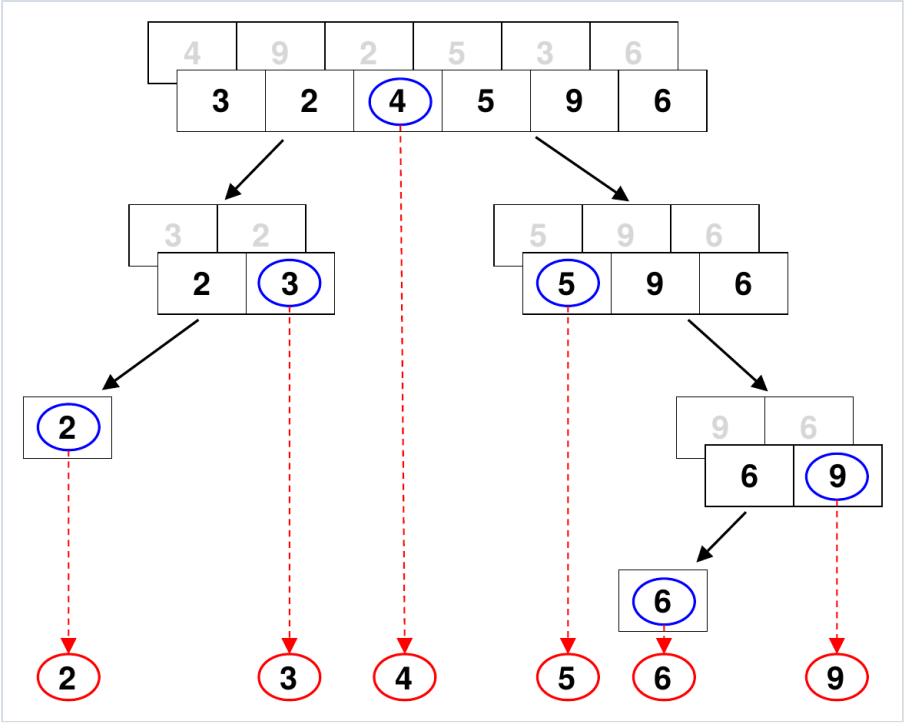
9.3 O quick sort passo a passo

Agora a lista inteira, `[4, 9, 2, 5, 3, 6]`, com o primeiro item como pivô (como nos quadros acima, é o caso em que o sorteio cai na primeira posição). Cada linha mostra uma chamada de `quick_sort` e a lista logo depois do particionamento daquela chamada. A coluna `Nível` mostra a profundidade da recursão (0 é a primeira chamada).

Passo	Nível	Chamada	Parte analisada	Pivô	Lista inteira depois	Posição do pivô / observação
1	0	`quick_sort(lista, 0, 5)`	`[4, 9, 2, 5, 3, 6]`	4	`[3, 2, 4, 5, 9, 6]`	2
2	1	`quick_sort(lista, 0, 1)`	`[3, 2]`	3	`[2, 3, 4, 5, 9, 6]`	1
3	2	`quick_sort(lista, 0, 0)`	`[2]`	-	`[2, 3, 4, 5, 9, 6]`	um único item (`2`): já está no lugar
4	2	`quick_sort(lista, 2, 1)`	`[]`	-	`[2, 3, 4, 5, 9, 6]`	parte vazia: nada a fazer
5	1	`quick_sort(lista, 3, 5)`	`[5, 9, 6]`	5	`[2, 3, 4, 5, 9, 6]`	3
6	2	`quick_sort(lista, 3, 2)`	`[]`	-	`[2, 3, 4, 5, 9, 6]`	parte vazia: nada a fazer
7	2	`quick_sort(lista, 4, 5)`	`[9, 6]`	9	`[2, 3, 4, 5, 6, 9]`	5
8	3	`quick_sort(lista, 4, 4)`	`[6]`	-	`[2, 3, 4, 5, 6, 9]`	um único item (`6`): já está no lugar
9	3	`quick_sort(lista, 6, 5)`	`[]`	-	`[2, 3, 4, 5, 6, 9]`	parte vazia: nada a fazer



Depois do primeiro particionamento: o `4` está no lugar e o quick sort recomeça nas duas sublistas.



Todas as chamadas: o pivô de cada chamada fica no lugar definitivo (setas vermelhas) e a lista termina ordenada.

Repare que as chamadas com **um item** ou com a **parte vazia** não fazem nada: é a condição de parada. E, a cada particionamento, mais um item (o pivô) fica definitivamente no seu lugar. Ao final, a lista está ordenada.

9.4 Implementação

```
        from random import randint


        def particionamento(lista, inicio, fim):
            indice = randint(inicio, fim)              # sorteia o índice do pivô
            aux = lista[inicio]                        # leva o pivô para a primeira
posição
            lista[inicio] = lista[indice]
            lista[indice] = aux

            pivo = lista[inicio]
            i = inicio                                 # i marca o fim da região dos
```

```
menores
            for j in range(inicio + 1, fim + 1):
                if lista[j] <= pivo:                    # achou um item menor ou igual ao
pivô
                    i += 1
                    aux = lista[i]                      # leva o item para a região dos
menores
                    lista[i] = lista[j]
                    lista[j] = aux

            aux = lista[inicio]                         # coloca o pivô entre as duas
regiões
            lista[inicio] = lista[i]
            lista[i] = aux
            return i                                    # devolve a posição final do pivô


        def quick_sort(lista, inicio, fim):
            if inicio < fim:                            # pelo menos 2 itens
                posicao = particionamento(lista, inicio, fim)
                quick_sort(lista, inicio, posicao - 1)  # lado esquerdo do pivô
                quick_sort(lista, posicao + 1, fim)     # lado direito do pivô
```

Linha a linha, em `particionamento`:

- `randint(inicio, fim)` sorteia um índice entre `inicio` e `fim`. As três linhas seguintes trocam o item sorteado com o da posição `inicio`.
- `pivo = lista[inicio]`: guarda o **valor** do pivô (que agora está na primeira posição). `i = inicio`: marca onde o pivô está.
- O `for` percorre `j` de `inicio + 1` até `fim` (por isso o `fim + 1` no `range`). Itens menores ou iguais ao pivô são levados para a esquerda.
- A última troca coloca o pivô na posição `i`, e `return i` informa essa posição.

Linha a linha, em `quick_sort`:

- `if inicio < fim`: só particiona se a parte tem pelo menos 2 itens (com 0 ou 1 item, não há o que fazer: é a condição de parada).
- Depois do particionamento, o pivô está na posição `posicao`. As duas chamadas recursivas ordenam o lado esquerdo (`inicio` até `posicao - 1`) e o lado direito (`posicao + 1` até `fim`).

```
        lista = [4, 9, 2, 5, 3, 6]
        quick_sort(lista, 0, len(lista) - 1)
        print(lista)                    # [2, 3, 4, 5, 6, 9]
```

9.5 Pontos de atenção

1) O pivô fica de fora das chamadas recursivas. Os intervalos são `inicio..posicao - 1` e `posicao + 1..fim`, sem a `posicao`, porque o pivô já está no lugar. Se o pivô entrasse em um dos lados, uma parte poderia continuar com o mesmo tamanho a cada chamada, e a recursão nunca terminaria (`RecursionError`), pelo mesmo motivo do `inicio = meio` da busca binária: o problema precisa ficar **menor**.

2) `randint` inclui os dois extremos. Diferente do `range`, o `randint(inicio, fim)` pode devolver o próprio `fim`. É o que queremos: o último item também pode ser o pivô.

3) `<=` leva os iguais para a esquerda. Itens iguais ao pivô ficam do lado esquerdo, e o pivô fica entre as duas partes.

4) Por que sortear o pivô? Com o pivô **fixo** (sempre o primeiro, por exemplo), uma lista **já ordenada** é o pior caso: o pivô é sempre o menor item, a parte esquerda fica vazia e a direita fica só um item menor. A recursão desce `N` níveis. Sorteando, é muito improvável ter sempre azar. (O exercício complementar 2 mostra o pior caso acontecendo.)

5) O quick sort não é estável. As trocas do particionamento levam itens para longe, e itens iguais podem trocar de ordem entre si. Com números isso não importa; com registros (produtos), pode importar.

9.6 Desempenho do quick sort

O desempenho depende das escolhas de pivô:

- **Melhor caso:** o pivô sempre cai no meio, dividindo a lista em duas partes iguais. Tempo da ordem de `N x log2(N)`.
- **Caso médio:** com pivôs sorteados, a divisão costuma ser razoável. Tempo também da ordem de `N x log2(N)`. É o caso que **mais acontece**.
- **Pior caso:** sucessivas péssimas escolhas de pivô (sempre o menor ou o maior item), que dividem a lista em partes de tamanho `0` e `N - 1`. Tempo da ordem de `N x N`. É raro, principalmente com o pivô sorteado.

Em cada chamada, o particionamento faz uma comparação para cada item da parte (menos o pivô). Diferente do merge sort, o quick sort ordena **no próprio lugar**, sem lista auxiliar.

10. Comparando na prática

Merge sort e quick sort têm tempo de execução parecido, e ambos são muito mais rápidos que os métodos simples em listas grandes. O merge sort é preferido quando a **estabilidade** é importante e quando se quer um tempo previsível (não depende da ordem dos itens). O quick sort é bastante usado por ordenar no lugar, sem memória extra.

Vamos **medir** com o `datetime` da Aula 13: o mesmo programa ordena uma lista aleatória de 20.000 números com cada método e mostra o tempo. Usamos os algoritmos desta aula, mais as funções prontas do Python (`lista.sort()` e `sorted(lista)`, detalhadas no tema complementar).

```
        from random import sample
        from datetime import datetime

        # (as funções bubble_sort, selection_sort, insertion_sort, merge_sort e
        # quick_sort da aula vêm antes, no mesmo arquivo)


        def criar_lista():
            """retorna uma lista aleatória com N itens"""
            n = 20000
            lista = sample(range(100000000), n)         # 20 mil números aleatórios
diferentes
            return lista


        lista = criar_lista()
        tempo_inicial = datetime.now()
        bubble_sort(lista)
        tempo_final = datetime.now()
        print(f'BubbleSort....: {tempo_final - tempo_inicial}')

        lista = criar_lista()
        tempo_inicial = datetime.now()
        selection_sort(lista)
        tempo_final = datetime.now()
        print(f'SelectionSort.: {tempo_final - tempo_inicial}')

        lista = criar_lista()
        tempo_inicial = datetime.now()
        insertion_sort(lista)
        tempo_final = datetime.now()
        print(f'InsertionSort.: {tempo_final - tempo_inicial}')

        lista = criar_lista()
        tempo_inicial = datetime.now()
        quick_sort(lista, 0, len(lista) - 1)
        tempo_final = datetime.now()
        print(f'QuickSort.....: {tempo_final - tempo_inicial}')

        lista = criar_lista()
        tempo_inicial = datetime.now()
        merge_sort(lista, 0, len(lista) - 1)
        tempo_final = datetime.now()
        print(f'MergeSort.....: {tempo_final - tempo_inicial}')

        lista = criar_lista()
```

```
        tempo_inicial = datetime.now()
        lista.sort()
        tempo_final = datetime.now()
        print(f'Sort Python...: {tempo_final - tempo_inicial}')

        lista = criar_lista()
        tempo_inicial = datetime.now()
        lista2 = sorted(lista)
        tempo_final = datetime.now()
        print(f'Sorted Python.: {tempo_final - tempo_inicial}')
```

Em cada medição, a lista é **criada de novo** (`criar_lista()`), para que todos os métodos comecem com uma lista embaralhada. Se a mesma lista fosse reaproveitada, o primeiro método a ordená-la deixaria a lista pronta para os seguintes.

Exemplo de saída (os tempos mudam de um computador para outro, e de uma execução para outra):

```
BubbleSort....: 0:00:26.984272
SelectionSort.: 0:00:22.098903
InsertionSort.: 0:00:16.384649
QuickSort.....: 0:00:00.045474
MergeSort.....: 0:00:00.072344
Sort Python...: 0:00:00.005816
Sorted Python.: 0:00:00.005081
```

O programa leva **alguns minutos** para terminar, porque os três métodos simples são lentos para 20.000 itens.

O que observar:

- Os três métodos simples levam **segundos ou dezenas de segundos**. O quick sort e o merge sort terminam em **centésimos de segundo**: centenas de vezes mais rápidos.
- Entre os três simples, o **bubble sort** costuma ser o mais lento (é o que mais comparações e trocas faz), mas a ordem entre selection e insertion pode mudar de um computador para outro, e até de uma execução para outra: as duas somam quase o mesmo número de operações (veja a tabela da comparação dos métodos simples).
- As funções prontas do Python (`sort` e `sorted`) foram as mais rápidas de todas. Elas são escritas em outra linguagem (C), mais veloz que o Python puro, e usam um algoritmo sofisticado (o **Timsort**, que combina ideias do merge sort e do insertion sort).

E o que acontece quando a lista **cresce**? Os tempos abaixo, medidos com listas de 5.000, 10.000 e 20.000 itens, mostram a diferença entre `N x N` e `N x log2(N)`:

Método	5.000 itens	10.000 itens	20.000 itens
Bubble sort	1,0971 s	4,2717 s	19,2196 s
Selection sort	0,6083 s	2,3333 s	8,5609 s
Insertion sort	0,7669 s	3,2389 s	13,5643 s
Merge sort	0,0092 s	0,0201 s	0,0413 s
Quick sort	0,0084 s	0,0162 s	0,0369 s
`sorted()` do Python	0,0007 s	0,0014 s	0,0033 s

(Medições feitas em um computador de teste; os seus números serão diferentes.)

Cada vez que o tamanho da lista **dobra**, os métodos simples levam cerca de **quatro vezes** mais tempo (`2 x 2`: é o efeito do `N x N`). Já o merge sort e o quick sort levam pouco mais que o **dobro** (o efeito do `N x log2(N)`). O `sorted()` do Python segue a mesma tendência dos sofisticados, só que mais rápido.

Cuidado com o tamanho da lista no teste. Uma versão comum deste programa cria a lista com `sample(list(range(100000000)), n)`. Isso monta antes uma lista com **100 milhões** de números, que ocupa vários gigabytes de memória (como o alerta da Aula 13 sobre 500 milhões de itens) e pode travar o computador. O `sample` aceita o `range` direto, sem criar a lista grande, e sorteia do mesmo jeito: `sample(range(100000000), n)`. Também vale começar com `n` menor (por exemplo, 5.000) se o computador for lento: com 20.000 itens, só os três métodos simples levam alguns minutos ao todo.

11. Exemplos de mercado

A ordenação está presente em muitos sistemas:

- **Rankings e placares:** ordenar jogadores ou vendedores pela pontuação, do maior para o menor.
- **Lojas virtuais:** a lista de produtos "do menor preço", "mais vendidos" ou "melhor avaliados" é uma ordenação por critério. Produtos com o mesmo valor mantêm a ordem anterior se o método for **estável**.
- **Bancos de dados:** o `ORDER BY` do SQL ordena o resultado de uma consulta, e os índices que aceleram buscas guardam dados ordenados.
- **Extratos e históricos:** movimentações ordenadas por data.
- **Linguagens de programação:** o `sort` e o `sorted` do Python usam o Timsort, um algoritmo que mistura merge sort e insertion sort.
- **Arquivos enormes:** quando os dados não cabem na memória, o método comum é ordenar pedaços menores e **intercalar** os resultados, a mesma ideia do merge sort.
- **Busca:** a busca binária da Aula 13 exige dados ordenados, então a ordenação é o primeiro passo.

12. Tema complementar: a ordenação pronta do Python (`sort`, `sorted`, `key` e `reverse`)

Na prática, o Python já traz ordenação pronta e muito rápida. Conhecer os algoritmos serve para entender o **custo** de ordenar e para saber o que acontece por baixo, mas em programas reais costuma-se usar as funções prontas. Existem duas:

```
numeros = [5, 3, 2, 1, 90, 6]

novos = sorted(numeros)          # devolve uma lista NOVA, ordenada
print(novos)                     # [1, 2, 3, 5, 6, 90]
print(numeros)                   # [5, 3, 2, 1, 90, 6]  (a original não mudou)

numeros.sort()                   # ordena a PRÓPRIA lista
print(numeros)                   # [1, 2, 3, 5, 6, 90]
```

	`sorted(lista)`	`lista.sort()`
Modifica a lista original?	Não	**Sim**
O que devolve?	Uma lista nova, ordenada	`None`
Aceita outros tipos (tupla, texto, conjunto)?	Sim (devolve sempre uma lista)	Não, só listas

Cuidado com o erro mais comum: `numeros = numeros.sort()` faz `numeros` virar `None`, pelo mesmo motivo do `bubble_sort` da seção 1.

12.1 `reverse`: ordem decrescente

```
print(sorted([5, 3, 2, 1, 90, 6], reverse=True))    # [90, 6, 5, 3, 2, 1]
```

12.2 `key`: ordenar por um critério

O parâmetro `key` recebe uma **função**. O Python chama essa função para **cada item**, e usa o valor devolvido para comparar. A lista continua guardando os itens originais; só a **comparação** usa o valor da `key`.

```
palavras = ["banana", "uva", "abacaxi", "kiwi"]

print(sorted(palavras))                 # ['abacaxi', 'banana', 'kiwi', 'uva']
(alfabética)
print(sorted(palavras, key=len))        # ['uva', 'kiwi', 'banana', 'abacaxi']
(pelo tamanho)
```

Outro exemplo, com maiúsculas e minúsculas (as maiúsculas vêm antes das minúsculas, Aula 05):

```
nomes = ["maria", "Ana", "bruno", "Carla"]

print(sorted(nomes))                    # ['Ana', 'Carla', 'bruno', 'maria']
print(sorted(nomes, key=str.lower))     # ['Ana', 'bruno', 'Carla', 'maria']
```

Atenção: `key=len` e `key=str.lower` vão **sem parênteses**. Com parênteses, a função seria executada na hora; sem eles, passamos a própria função para o Python chamá-la em cada item.

Para itens mais complexos, escreve-se uma função própria. Para ordenar uma lista de dicionários pelo preço:

```
def preco_do_produto(produto):
    return produto["preco"]


produtos = [
    {"nome": "Camiseta", "preco": 59.90},
    {"nome": "Boné", "preco": 39.90},
    {"nome": "Tênis", "preco": 199.90},
]

for produto in sorted(produtos, key=preco_do_produto):
    print(produto["nome"], produto["preco"])
# Boné 39.9
# Camiseta 59.9
# Tênis 199.9
```

(Existe uma forma mais curta de escrever funções de uma linha, a função `lambda`: `key=lambda produto: produto["preco"]`. Aparece muito em código real.)

12.3 Estabilidade

O `sort` e o `sorted` do Python são **estáveis**: itens que **empatam** na comparação mantêm a ordem em que já estavam. Isso permite ordenar por **dois critérios**: ordena-se primeiro pelo critério **menos importante** e depois pelo **mais importante**.

```
alunos = [("Carlos", 8), ("Ana", 9), ("Bruno", 8), ("Diana", 9)]

def nome_do_aluno(aluno):
    return aluno[0]


def nota_do_aluno(aluno):
    return aluno[1]
```

```
        alunos = sorted(alunos, key=nome_do_aluno)                # 1º: critério de
desempate (nome)
        alunos = sorted(alunos, key=nota_do_aluno, reverse=True)  # 2º: critério principal
(nota)
        print(alunos)
        # [('Ana', 9), ('Diana', 9), ('Bruno', 8), ('Carlos', 8)]
```

As maiores notas vêm primeiro, e quem empata na nota está em ordem alfabética, porque a primeira ordenação (por nome) foi **preservada** nos empates da segunda.

13. Exercícios principais

Os exercícios abaixo estão em ordem **crescente de dificuldade**. Os algoritmos da aula (`bubble_sort`, `selection_sort`, `insertion_sort`, `merge_sort` e `quick_sort`) podem ser usados como ponto de partida.

Exercício 1: Ordem decrescente

Implemente a função `bubble_sort_decrescente(lista)`, alterando o bubble sort para que ele ordene a lista do **maior para o menor** item.

Por exemplo, `[5, 3, 2, 1, 90, 6]` deve ficar `[90, 6, 5, 3, 2, 1]`.

Dica: o que muda no algoritmo é só uma coisa. Pense: em que situação dois vizinhos estão **fora de ordem** quando se quer a ordem decrescente?

Exercício 2: Melhor caso, 1000 itens em ordem crescente

Preencha uma lista com **1000 números inteiros em ordem CRESCENTE**. Ordene a lista utilizando os métodos **bubble sort, selection sort e insertion sort**. Informe, para cada método:

- a quantidade de operações de **comparações** entre elementos;
- a quantidade de operações de **trocas** entre elementos.

Dicas:

- Crie uma versão de cada método que **conte** as operações e devolva as duas quantidades.
- Antes de executar, **preveja** os números usando a tabela de comparação dos métodos simples (para `N = 1000`), e depois confira.
- Cuidado com **onde** o contador de comparações do insertion sort é incrementado: a comparação acontece a cada volta do `while`.

Exercício 3: Pior caso, 1000 itens em ordem decrescente

Repita o exercício anterior, agora com uma lista de **1000 números inteiros em ordem DECRESCENTE** (1000, 999, ..., 1). Reaproveite as funções com contagem do exercício 2. Informe, para cada método, as comparações e as trocas e compare com os resultados do exercício 2: o que mudou? O que **não** mudou?

Exercício 4: Bubble sort com parada antecipada

Implemente `bubble_sort_otimizado(lista)`, uma versão do bubble sort que **para** assim que uma varredura termina sem fazer nenhuma troca (pois então a lista já está ordenada). A função deve devolver a **quantidade de comparações** realizadas.

Teste com uma lista de 1000 itens já ordenada, com uma de 1000 itens em ordem decrescente e com `[5, 3, 2, 1, 90, 6]`. Compare com a quantidade de comparações do bubble sort comum.

Dica: use uma variável para "lembrar" se houve alguma troca na varredura atual. Onde ela deve ser reiniciada?

Exercício 5: Ordenando por um critério (produtos por preço)

Implemente `insertion_sort_por_campo(lista, campo)`, que ordena, em ordem **crescente**, uma lista de **dicionários** pelo valor de um campo, usando o insertion sort. Por exemplo, com produtos no formato `{"nome": "Boné", "preco": 39.90}`, ordenar por `"preco"` ou por `"nome"`.

Teste com esta lista, ordenando por preço e depois por nome:

```
produtos = [
    {"nome": "Camiseta", "preco": 59.90},
    {"nome": "Tênis", "preco": 199.90},
    {"nome": "Boné", "preco": 39.90},
    {"nome": "Mochila", "preco": 149.90},
    {"nome": "Meia", "preco": 39.90},
]
```

Repare no que acontece com o Boné e a Meia, que têm o mesmo preço.

Exercício 6: Intercalando duas listas ordenadas

Escreva a função `intercalar_listas(lista_a, lista_b)`, que recebe **duas listas já ordenadas** (em ordem crescente) e devolve uma **nova lista**, também ordenada, com todos os itens das duas. As listas recebidas não devem ser modificadas.

Por exemplo, `intercalar_listas([1, 4, 9], [2, 3, 10, 11])` devolve `[1, 2, 3, 4, 9, 10, 11]`. Teste também com uma das listas vazia.

Essa é a ideia central do merge sort, sem índices `inicio`, `meio` e `fim`. Dica: como as duas listas já estão ordenadas, só é preciso olhar o primeiro item ainda não usado de cada uma.

Exercício 7: Contando comparações: merge sort e quick sort x insertion sort

Crie versões do **merge sort** e do **quick sort** que devolvam a **quantidade de comparações** feitas (`merge_sort_comparacoes` e `quick_sort_comparacoes`). Depois, ordene **a mesma lista aleatória de 2000 números** com o insertion sort (versão com contagem do exercício 2), com o merge sort e com o quick sort, e compare as quantidades de comparações.

Dicas:

- No merge sort, só a intercalação compara itens. No quick sort, só o particionamento.
- Como as funções são recursivas, o total de uma chamada é o que ela contou **mais** o que as chamadas menores contaram (como na soma de lista da Aula 12).
- Cada método deve receber uma **cópia** da lista (`lista.copy()`). Por quê?
- Para ter sempre a mesma lista nos testes, use `random.seed(42)` antes de sortear com `random.sample(range(1000000), 2000)`.

14. Exercícios complementares

Os exercícios abaixo são **complementares**: reforçam os algoritmos da aula com testes e com uma variação do quick sort.

Exercício complementar 1: A lista está ordenada? (e testes automáticos)

a) Escreva a função `esta_ordenada(lista)`, que devolve `True` se a lista está em ordem crescente (itens iguais são permitidos) e `False` caso contrário.

b) Escreva `testar_algoritmos(repeticoes=100)`, que gera listas aleatórias pequenas (de 0 a 30 itens, com valores repetidos, inclusive listas vazias), ordena uma cópia com cada um dos 5 algoritmos da aula e confere se o resultado está ordenado **e** é igual ao devolvido por `sorted()`. Devolve `True` se todos passaram em todos os testes.

Exercício complementar 2: Quick sort com o último elemento como pivô

Implemente `particionamento_ultimo(lista, inicio, fim)` e `quick_sort_ultimo(lista, inicio, fim)`, uma versão do quick sort em que o pivô é **sempre o último elemento** da parte analisada (sem sorteio). Teste com uma lista embaralhada de 2000 itens e, depois, com `list(range(2000))`, que já está ordenada. O que acontece? Por quê?

Dica: o pivô começa na posição `fim`. Percorra de `inicio` até `fim - 1` e, no final, coloque o pivô entre as duas regiões.

15. Exercícios do tema complementar: `sort`, `sorted`, `key` e `reverse`

Os exercícios abaixo usam o que foi visto na seção 12.

Exercício do tema complementar 1: Nomes em ordem decrescente, sem diferenciar maiúsculas

Dada a lista `nomes = ["maria", "Ana", "bruno", "Carla", "ana"]`, mostre:

a) a lista em ordem alfabética **simples** (sem `key`); b) a lista em ordem **decrescente**, sem diferenciar maiúsculas de minúsculas; c) uma cópia da lista ordenada com `.sort()`, confirmando que a lista original **não** foi alterada (use `.copy()`).

Explique por que `"Ana"` e `"ana"` aparecem na ordem em que aparecem.

Exercício do tema complementar 2: Catálogo ordenado por preço

Dado um catálogo no formato do projeto (dicionário de dicionários, com o código do produto como chave), exiba os produtos do **mais caro ao mais barato** usando `sorted`, `key` e `reverse`. Use um catálogo de exemplo como este:

```
catalogo_exemplo = {
    "CAM13": {"nome": "Camiseta Azul", "preco": 59.90, "estoque": 120},
    "TÊN12": {"nome": "Tênis Runner", "preco": 199.90, "estoque": 30},
    "BON10": {"nome": "Boné Preto", "preco": 39.90, "estoque": 50},
    "MOC13": {"nome": "Mochila Cinza", "preco": 149.90, "estoque": 15},
}
```

Dica: `catalogo_exemplo.items()` produz tuplas `(codigo, dados)`. A função `key` recebe uma dessas tuplas.

Exercício do tema complementar 3: Dois critérios de ordenação

Exiba os produtos do catálogo abaixo por **estoque, do maior para o menor** e, em caso de empate no estoque, por **nome, em ordem alfabética**. Use a estabilidade do `sorted`.

```
catalogo_empate = {
    "CAM13": {"nome": "Camiseta Azul", "preco": 59.90, "estoque": 10},
    "TÊN12": {"nome": "Tênis Runner", "preco": 199.90, "estoque": 30},
    "BON10": {"nome": "Boné Preto", "preco": 39.90, "estoque": 10},
    "MOC13": {"nome": "Mochila Cinza", "preco": 149.90, "estoque": 30},
}
```

Dica: em qual ordem fazer as duas ordenações?

16. Evolução do projeto de fundo: E-commerce simples

Assim como nas Aulas 08 a 13, a Aula 14 apenas **adiciona** funcionalidades novas ao projeto, sem alterar nenhuma função que já existia.

Na aula passada, o índice por nome foi ordenado com a função `sorted()` do Python, porque ainda não conhecíamos os algoritmos de ordenação. Agora que conhecemos, o projeto ganha:

- uma forma de **ver o catálogo ordenado** por nome, preço ou estoque, usando o **insertion sort** (um catálogo de loja pequena é o tipo de conjunto em que os métodos simples servem bem, e o insertion sort é estável: produtos com o mesmo valor ficam na ordem em que estavam);
- o **índice por nome** gerado com o **merge sort**, com o mesmo resultado do índice da Aula 13.

O catálogo é um **dicionário**, e um dicionário não tem posições para trocar. Por isso, as funções de ordenação trabalham com uma **lista** de tuplas `(codigo, dados)` montada a partir de `catalogo.items()`. O catálogo original **não é modificado**.

Enunciado

1. Crie `ordenar_produtos_por_campo`, que recebe o catálogo, o `campo` (`"nome"`, `"preco"` ou `"estoque"`, com valor padrão `"preco"`) e o parâmetro `decrescente` (valor padrão `False`). Devolve a lista de tuplas `(codigo, dados)` ordenada pelo campo, usando o **insertion sort**. O nome deve ser comparado em minúsculas. Se o campo for inválido, lance `ValueError` (Aula 08).
2. Crie `exibir_catalogo_ordenado`, com os mesmos parâmetros, que mostra o catálogo na ordem pedida, **reaproveitando** `exibir_catalogo` (transforme a lista em dicionário com `dict()`: os dicionários guardam a ordem de inserção).
3. Crie `intercala` e `merge_sort` (os da aula) e a função `gerar_indice_por_nome_com_merge_sort`, que devolve o mesmo índice de `gerar_indice_por_nome`, mas ordenado com o **merge sort**.
4. Na demonstração, exiba o catálogo por preço, por estoque (decrescente) e por nome; mostre a recusa de um campo inválido; confira que o índice do merge sort é igual ao índice com `sorted()` e que a busca binária funciona com ele. Por fim, ordene um catálogo de 2000 produtos de três jeitos e compare os tempos.

Atenção a dois detalhes:

- A lista ordenada guarda os **mesmos dicionários** `dados` do catálogo (não cópias). Se alguém alterar o `estoque` de um produto pela lista ordenada, o catálogo muda também.
- Assim como o índice da Aula 13, o resultado é uma **foto** do momento: um produto cadastrado depois não aparece na lista ordenada já gerada.